Ride-Hailing Dispatch System



Kelompok: 3

Anggota Kelompok:

Aullya Nadine Kuswandi - 2802405125

Ellyn Auria - 2802396096

Felice Yang - 2802406456

Jaiden Ananta Halim - 2802397615

Raffael Gabriel Rei Lumbantobing - 2602191043

Big Data Infrastructure and Technology

Kelas: LA09

2025/2026

BAB I PENDAHULUAN

A. Latar Belakang

Perkembangan teknologi digital dan penggunaan perangkat *mobile* yang semakin luas telah mengubah berbagai aspek kehidupan masyarakat, termasuk dalam sektor transportasi. Salah satu inovasi yang berkembang pesat adalah layanan *ride-hailing*, yaitu layanan transportasi berbasis aplikasi yang memungkinkan pengguna memesan kendaraan secara online dengan mudah dan cepat. Dengan memanfaatkan teknologi Global Positioning System (GPS), aplikasi *ride-hailing* dapat menghubungkan penumpang dengan pengemudi secara *real-time* sehingga meningkatkan efisiensi dan kenyamanan dalam mobilitas sehari-hari.

Seiring dengan meningkatnya jumlah pengguna, aplikasi *ride-hailing* harus mampu menangani jutaan permintaan perjalanan, pemrosesan lokasi secara *real-time*, sistem pembayaran, serta berbagai fitur pendukung lainnya. Kondisi ini menuntut infrastruktur teknologi yang memiliki tingkat ketersediaan tinggi, skalabilitas yang baik, keamanan data yang memadai, dan kemampuan pemrosesan data secara cepat. Infrastruktur tradisional seringkali menghadapi keterbatasan dalam memenuhi kebutuhan tersebut, terutama ketika terjadi lonjakan pengguna pada waktu-waktu tertentu.

Cloud computing menjadi salah satu solusi yang banyak digunakan untuk mengatasi tantangan tersebut. Amazon Web Services (AWS) menyediakan berbagai layanan *cloud* yang dapat dimanfaatkan untuk membangun sistem *ride-hailing* yang andal, fleksibel, dan efisien. Melalui layanan AWS yang tersedia, aplikasi dapat dirancang untuk mendukung kebutuhan operasional skala besar dengan tetap menjaga performa dan ketersediaan layanan.

Oleh karena itu, pada laporan ini akan dirancang arsitektur sistem *ride-hailing* berbasis AWS yang mampu memenuhi kebutuhan fungsional dan non-fungsional aplikasi, termasuk pencocokan pengemudi dan penumpang, pemrosesan data lokasi secara *real-time*, serta aspek keamanan, skalabilitas, dan keandalan sistem.

B. Tujuan

Tujuan yang ingin dicapai pada proyek ini adalah:

1. Menghasilkan desain arsitektur cloud untuk *ride-hailing dispatch system* yang mampu memproses *ingestion* data lokasi dan permintaan pengguna secara *real-time*.

2. Membangun infrastruktur yang memiliki kemampuan *high availability* untuk menjamin ketersediaan layanan bagi pengguna.
3. Mengimplementasikan sumber daya yang dinamis untuk memastikan performa sistem tetap optimal di berbagai kondisi beban kerja dengan biaya yang tetap efisien.

C. Rumusan Masalah

Rumusan masalah dalam proyek ini adalah:

1. Bagaimana arsitektur *cloud* yang mampu menangani pengiriman data GPS secara *real-time* dari banyak pengemudi dengan latensi yang sangat rendah?
2. Bagaimana arsitektur *cloud* yang memiliki kemampuan *high availability* dan *fault tolerance* agar sistem *dispatch* tetap beroperasi tanpa gangguan?
3. Bagaimana mengkonfigurasikan mekanisme *auto-scaling* agar infrastruktur dapat beradaptasi secara efisien terhadap lonjakan *workload* pada jam sibuk sekaligus mengoptimalkan efisiensi biaya saat lalu lintas sepi?

BAB II RIDE-HAILING DISPATCH SYSTEM

A. Gambaran dan Mekanisme Sistem

Sistem *ride-hailing* ini dirancang untuk melayani skala operasional yang masif, dengan estimasi Pengguna Aktif Harian (*Daily Active Users*) mencapai 250.000 pengguna. Infrastruktur komputasi diproyeksikan untuk menghadapi fluktuasi beban kerja yang signifikan setiap harinya. Titik puncak operasional (*maximum hit*) diproyeksikan terjadi pada jam sibuk komuter, yakni rentang pukul 07:00–09:00 dan 16:00–19:00, di mana permintaan layanan (ingestion data GPS dan pesanan) mencapai batas maksimum. Sebaliknya, titik beban terendah (*minimum hit*) akan terjadi pada dini hari, sekitar pukul 01:00–04:00. Untuk mengakomodasi dinamika operasional tersebut secara efisien, sistem ini dibangun atas empat mekanisme utama:

1. Pengumpulan Data Lokasi

Sistem *ride-hailing* bergantung pada informasi lokasi yang akurat untuk dapat melakukan pencocokan antara penumpang dan pengemudi. Oleh karena itu, aplikasi penumpang dan pengemudi secara berkala mengirimkan data lokasi yang diperoleh dari GPS perangkat ke *backend system*. Data ini mencakup koordinat geografis, waktu pembaruan lokasi, serta status pengguna atau pengemudi. Informasi lokasi yang diperbarui secara *real-time* memungkinkan sistem mengetahui posisi terkini pengemudi yang sedang aktif sehingga proses pencarian pengemudi terdekat dapat dilakukan dengan cepat dan akurat. Selain itu, data lokasi juga digunakan untuk menghitung estimasi waktu penjemputan dan memantau perjalanan yang sedang berlangsung.

2. Algoritma Pencocokan Penumpang dan Pengemudi

Setelah permintaan perjalanan diterima, sistem dispatch akan mencari pengemudi yang tersedia di sekitar lokasi penjemputan. Pencarian biasanya dilakukan dalam radius tertentu yang dapat disesuaikan berdasarkan tingkat kepadatan pengemudi dan kondisi operasional pada wilayah tersebut. Selanjutnya, algoritma dispatch melakukan evaluasi terhadap kandidat pengemudi yang ditemukan. Beberapa faktor yang dapat dipertimbangkan antara lain jarak pengemudi ke titik penjemputan, estimasi waktu kedatangan, kondisi lalu lintas, tingkat aktivitas pengemudi, serta status ketersediaannya saat itu. Berdasarkan hasil evaluasi tersebut, sistem memilih pengemudi yang

dianggap paling optimal untuk melayani permintaan perjalanan. Tujuan utama dari proses ini adalah meminimalkan waktu tunggu penumpang sekaligus meningkatkan efisiensi penggunaan armada pengemudi yang tersedia.

3. **Notifikasi dan Konfirmasi Perjalanan**

Setelah pengemudi terpilih, sistem mengirimkan notifikasi secara *real-time* ke aplikasi pengemudi yang berisi detail permintaan perjalanan. Pengemudi diberikan waktu tertentu untuk menerima atau menolak permintaan tersebut. Apabila pengemudi menerima permintaan, sistem akan mengonfirmasi kecocokan antara penumpang dan pengemudi, kemudian mengirimkan informasi masing-masing pihak seperti nama, lokasi, dan estimasi waktu penjemputan. Namun apabila pengemudi menolak atau tidak memberikan respons hingga batas waktu yang ditentukan, sistem secara otomatis mengalihkan permintaan tersebut kepada kandidat pengemudi berikutnya tanpa memerlukan intervensi manual. Mekanisme ini memungkinkan proses dispatch tetap berjalan secara efisien meskipun terdapat pengemudi yang tidak dapat menerima perjalanan.

4. **Pembayaran Layanan**

Ketika perjalanan selesai, sistem menghitung total biaya yang harus dibayarkan oleh pengguna. Perhitungan tarif dapat mempertimbangkan berbagai faktor seperti jarak tempuh, durasi perjalanan, jenis layanan yang dipilih, ataupun biaya tambahan tertentu. Setelah biaya dihitung, sistem memproses pembayaran sesuai metode yang dipilih pengguna. Pembayaran dapat dilakukan secara tunai maupun melalui layanan pembayaran digital. Status transaksi kemudian diperbarui dan disimpan sebagai bagian dari riwayat perjalanan pengguna.

5. **Penyimpanan Data**

Sistem *ride-hailing* menghasilkan berbagai jenis data selama operasional berlangsung, seperti data pengguna, data pengemudi, lokasi GPS, detail perjalanan, dan informasi transaksi pembayaran. Seluruh data tersebut disimpan secara terpusat agar dapat diakses kembali ketika diperlukan untuk mendukung operasional layanan, autentikasi pengguna, pengelolaan profil, penyimpanan riwayat perjalanan, serta pencatatan transaksi. Selain itu, sistem harus memastikan data tetap aman, konsisten, dan tersedia melalui mekanisme seperti backup data, replikasi database, dan pengendalian akses yang memadai

B. Kebutuhan Non-Fungsional Sistem

Untuk menopang operasional *ride-hailing* skala nasional dengan proyeksi lalu lintas data yang telah dijabarkan, infrastruktur dituntut untuk beroperasi pada standar *enterprise-grade*. Oleh karena itu, sistem harus memenuhi sejumlah kebutuhan non-fungsional guna menjamin kualitas layanan, ketersediaan sistem, serta keamanan operasional. Kebutuhan tersebut diantaranya:

1. **Availability (Ketersediaan)**

   Sistem harus memiliki tingkat ketersediaan yang tinggi karena layanan *ride-hailing* digunakan sepanjang waktu. Kegagalan sistem dispatch dapat menyebabkan pengguna tidak dapat melakukan pemesanan atau pengemudi tidak menerima permintaan perjalanan. Oleh karena itu, diperlukan mekanisme redundansi dan failover untuk menjaga layanan tetap berjalan meskipun terjadi gangguan pada sebagian komponen sistem.

2. **Low Latency (Latensi Rendah)**

   Proses pencocokan antara penumpang dan pengemudi harus dilakukan dengan latensi yang rendah agar pengguna memperoleh respons dalam hitungan detik. Pembaruan lokasi GPS dan pengiriman notifikasi juga harus dilakukan secara *real-time* untuk memastikan pengalaman pengguna yang baik.

3. **Scalability (Skalabilitas)**

   Sistem harus mampu menangani lonjakan jumlah pengguna dan pengemudi, terutama pada jam sibuk atau saat terjadi event tertentu. Arsitektur sistem perlu mendukung horizontal scaling sehingga kapasitas layanan dapat ditingkatkan dengan menambahkan sumber daya komputasi tanpa mengganggu layanan yang sedang berjalan.

4. **Security (Keamanan)**

   Data lokasi pengguna dan pengemudi merupakan informasi sensitif yang harus dilindungi. Sistem harus menerapkan enkripsi data saat transmisi maupun penyimpanan, autentikasi pengguna dan pengemudi, serta kontrol akses yang memadai untuk mencegah akses tidak sah dan kebocoran data.

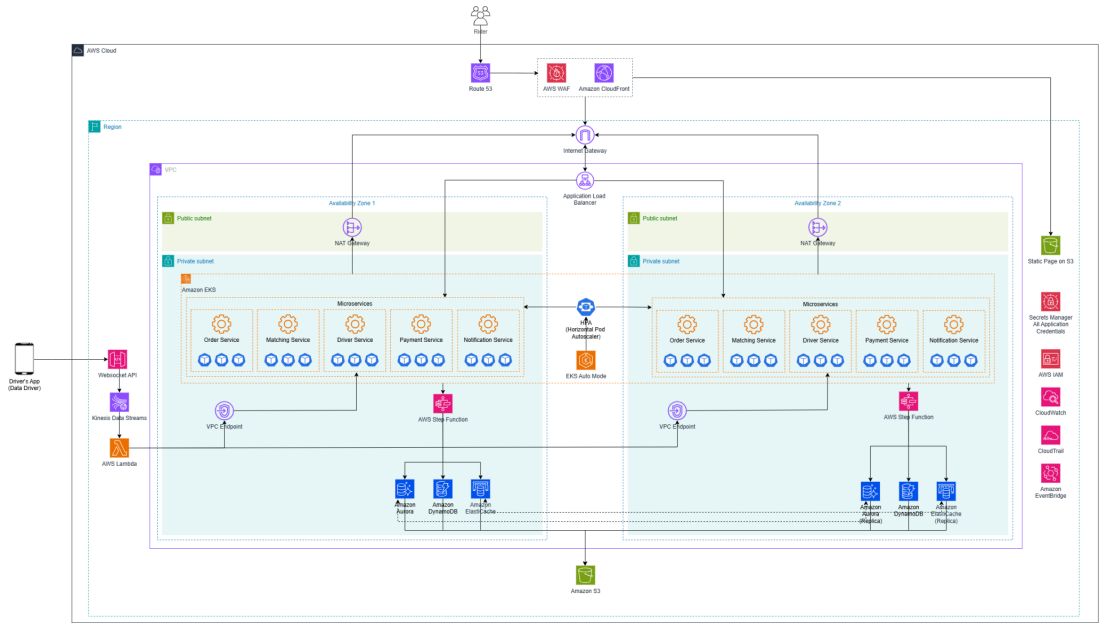
5. **Consistency and Accuracy (Konsistensi dan Akurasi)**

   Sistem harus memastikan bahwa informasi lokasi, status pengemudi, dan status perjalanan selalu akurat dan konsisten. Ketidaksesuaian data dapat

menyebabkan kesalahan pencocokan, pengiriman pengemudi yang salah, atau pengalaman pengguna yang buruk.

BAB III PERANCANGAN ARSITEKTUR CLOUD

A. Arsitektur Cloud *Ride-Hailing* Dispatch System



B. Arsitektur Jaringan

1. Amazon Virtual Private Cloud (VPC)

Amazon Virtual Private Cloud (VPC) adalah jaringan virtual terisolasi secara logis yang disediakan di dalam infrastruktur cloud publik. VPC memungkinkan pengguna untuk meluncurkan sumber daya AWS ke dalam jaringan virtual yang terisolasi secara privat dengan kontrol penuh atas konfigurasi alamat IP, *routing tables*, dan *network gateways*. Pada arsitektur *ride-hailing*, VPC berfungsi sebagai batas keamanan (*security boundary*) yang mengisolasi komponen-komponen dari akses tidak sah melalui jaringan publik.

2. Internet Gateway (IGW)

Internet Gateway (IGW) adalah sistem yang berfungsi sebagai pintu gerbang penghubungan antara jaringan lokal dengan jaringan global. Internet gateway merupakan komponen VPC yang bersifat *highly available* dan berfungsi menjembatani komunikasi antar sumber daya di dalam VPC dengan jaringan internet luar. IGW memfasilitasi translasi alamat jaringan (*Network Address Translation*) untuk pemetaan lalu lintas data. Pada arsitektur *ride-hailing*, Internet Gateway diposisikan sebagai pintu gerbang utama

setelah lalu lintas data melewati lapisan keamanan AWS WAF dan Amazon CloudFront. Komponen ini bertindak sebagai perantara yang menerima permintaan masuk dari aplikasi penumpang dan kemudian diteruskan ke sistem internal AWS.

3. **NAT Gateway**

NAT Gateway merupakan layanan *Network Address Translation* (NAT) yang terkelola oleh AWS. NAT Gateway wajib ditempatkan di *public subnet* yang berfungsi mengizinkan sumber daya di dalam *private subnet* terhubung ke internet luar, namun secara ketat memblokir koneksi masuk oleh internet luar. Pada arsitektur *ride-hailing*, NAT Gateway bertindak sebagai jembatan antara keamanan internal sistem dan fungsionalitas eksternal di internet publik. Komponen *microservices* berada pada *private subnet* agar terisolasi dari potensi serangan luar. Namun, dalam operasi sehari-hari, komponen tersebut tidak bekerja secara mandiri dan wajib melakukan komunikasi keluar. NAT Gateway berfungsi menerima lalu lintas data dari *private subnet* dan mengubah IP privat server menjadi Elastic IP (IP publik statis) lalu meneruskannya ke internet luar. Melalui mekanisme ini, pihak luar tidak dapat masuk ke server sementara server internal dapat mengakses sumber daya di internet.

4. **Multi-Availability Zone (Multi-AZ)**

Availability Zone (AZ) adalah salah satu atau beberapa *data center* yang terisolasi secara redundan di dalam satu region. Strategi multi-AZ melibatkan replikasi dan penyebaran infrastruktur aplikasi di beberapa AZ untuk memastikan ketersediaan tinggi (*high availability*) dan toleransi kesalahan (*fault tolerance*). Pada arsitektur *ride-hailing*, AZ dibagi menjadi AZ 1 dan AZ 2 karena karakteristik layanan *ride-hailing* yang beroperasi harian tanpa henti. Pembagian ini memastikan jika terjadi kegagalan dalam sistem atau gangguan fisik pada salah satu AZ, lalu lintas data dan transaksi akan dialihkan secara instan ke AZ replika yang aktif tanpa menyebabkan *downtime* bagi pengguna.

5. **Application Load Balancer (ALB)**

Application Load Balancer (ALB) merupakan salah satu fitur dari *Elastic Load Balancing* yang beroperasi pada *application layer* dalam model *Open Systems Interconnection* (OSI). ALB bertugas mendistribusikan lalu

lintas aplikasi yang masuk secara otomatis ke beberapa target di berbagai AZ berdasarkan permintaan. Pada arsitektur *ride-hailing*, ALB diposisikan secara terpusat untuk mendistribusikan beban kerja secara merata ke *cluster microservices* pada Availability Zone 1 dan Availability Zone 2. Saat terjadi lonjakan permintaan dari jutaan penumpang, ALB mengoptimalkan kinerja sistem dengan memastikan tidak ada *cluster* pada Amazon EKS yang mengalami kelebihan beban (*overload*).

6. VPC Endpoint

VPC Endpoint adalah layanan AWS yang memungkinkan resource di dalam sebuat Amazon VPC berkomunikasi dengan layanan AWS lainnya tanpa harus melewati Internet Gateway, NAT Gateway, atau internet publik. Pada arsitektur *ride-hailing* ini, VPC Endpoint berfungsi sebagai jalur komunikasi privat yang menghubungkan data lokasi dan status *driver* yang diproses oleh AWS Lambda dengan *microservices* Amazon EKS yang berada di private subnet sehingga pertukaran data *real-time* dapat berlangsung secara aman, latensi rendah, dan tanpa melewati internet publik.

C. Service AWS

1. Amazon Route 53

Amazon Route 53 merupakan layanan DNS (*Domain Name System*) yang menyediakan ketersediaan tinggi (*highly available*) dan terukur (*scalable*). Amazon Route 53 digunakan untuk menerjemahkan nama domain menjadi alamat IP sehingga pengguna dapat mengakses dengan mudah. Route 53 juga berfungsi untuk mengelola lalu lintas jaringan (*traffic routing*) dan memantau kesehatan server (*health checking*).

Alasan penggunaan Amazon 53 pada arsitektur *ride-hailing*:

a. *High Availability* dan *Traffic Routing*

Route 53 mengarahkan seluruh permintaan pengguna ke Amazon CloudFront berdasarkan konfigurasi DNS yang telah ditentukan. Sebelum diproses lebih lanjut, lalu lintas akan diperiksa oleh AWS WAF untuk memfilter permintaan yang berpotensi berbahaya.

b. *Health Checking*

Route 53 secara berkala melakukan pemeriksaan kesehatan terhadap endpoint yang telah dikonfigurasikan. Apabila endpoint utama tidak sehat, Route 53 akan mengaktifkan mekanisme *DNS Failover* untuk mengalihkan lalu lintas ke endpoint cadangan.

2. AWS Web Application Firewall (WAF)

AWS Web Application Firewall (WAF) merupakan layanan keamanan yang berfungsi melindungi aplikasi dan API dari berbagai serangan yang berasal dari internet pada *application layer* dalam model OSI. AWS WAF bekerja dengan memeriksa setiap permintaan HTTP/HTTPS yang masuk sebelum permintaan tersebut diteruskan ke layanan backend seperti Amazon CloudFront, ALB, atau API Gateway. Pada arsitektur *ride-hailing,* AWS WAF diintegrasikan dengan Amazon CloudFront sehingga seluruh lalu lintas dari pengguna akan diperiksa terlebih dahulu sebelum mencapai aplikasi.

Alasan penggunaan AWS WAF pada arsitektur *ride-hailing*:

a. Melindungi dari SQL Injection (SQLi)

SQL Injection merupakan serangan yang mencoba memanipulasi query database melalui input pengguna. Jika tidak difilter, penyerang dapat memperoleh akses data pengguna atau memanipulasi database. AWS WAF dapat mendeteksi pola serangan ini dan langsung memblokirnya sehingga data penting seperti data penumpang, data *driver*, riwayat perjalanan, dan informasi pembayaran tidak akan bocor.

b. Mengurangi Beban Backend

AWS WAF diintegrasikan langsung oleh Amazon CloudFront untuk memberikan perlindungan terhadap aplikasi sejak permintaan pertama kali masuk ke jaringan AWS. Ketika pengguna mengakses aplikasi, permintaan akan diterima oleh *edge location* CloudFront terdekat. Permintaan yang berbahaya langsung diblokir oleh AWS WAF sehingga layanan AWS lainnya tidak perlu memproses permintaan tersebut. Hal ini dapat menghemat sumber daya komputasi dan meningkatkan performa aplikasi.

c. Dikelola Sepenuhnya oleh AWS

AWS WAF merupakan layanan yang sepenuhnya dikelola oleh AWS sehingga pengguna tidak perlu melakukan instalasi, konfigurasi infrastruktur, maupun pemeliharaan firewall secara manual. AWS bertanggung jawab terhadap ketersediaan layanan, pembaruan sistem, skalabilitas, serta pengelolaan infrastruktur yang mendasarinya. Pengguna hanya perlu menentukan aturan keamanan (*security rules*) yang diinginkan melalui AWS Management Console atau AWS CLI. Selain itu, AWS juga menyediakan berbagai *AWS Managed Rules* yang secara rutin diperbarui untuk melindungi aplikasi dari ancaman keamanan.

3. Amazon CloudFront

Amazon CloudFront merupakan layanan CDN (Content Delivery Network) yang digunakan untuk mempercepat distribusi konten web statis dan dinamis kepada pengguna melalui jaringan global *edge location* milik AWS. Ketika pengguna mengakses aplikasi, permintaan akan diarahkan ke *edge location* CloudFront yang memiliki latensi terendah. Jika konten yang diminta sudah tersedia di *cache* CloudFront, konten tersebut akan langsung dikirim kepada pengguna. Jika belum tersedia, CloudFront akan mengambil konten dari origin yang telah dikonfigurasi, kemudian menyimpannya di *cache* untuk permintaan berikutnya.

Alasan penggunaan Amazon CloudFront pada arsitektur *ride-hailing*:

a. Menurunkan Latensi Aplikasi

Amazon CloudFront membantu menurunkan latensi aplikasi dengan cara mendistribusikan konten ke berbagai lokasi di seluruh dunia. Ketika seseorang menggunakan aplikasi *ride-hailing*, permintaannya akan diarahkan ke *edge location* yang paling dekat dengan tempat mereka berada. Jika konten yang dibutuhkan sudah tersedia di *cache*, maka CloudFront dapat langsung mengirimkan konten tersebut tanpa perlu mengambilnya dari server utama di AWS Region. Dengan cara ini, jarak yang harus ditempuh oleh data menjadi lebih pendek dan waktu respons aplikasi menjadi lebih cepat.

b. Mengurangi Beban ALB dan Amazon EKS

CloudFront membantu mengurangi pekerjaan yang dilakukan oleh ALB dan Amazon EKS dengan cara menyimpan konten yang sering dibuka, seperti halaman statis, gambar, kode JavaScript, CSS, atau aset frontend lainnya, di dalam *cache* CloudFront. Ketika seseorang mengakses konten yang sama, CloudFront akan langsung memberikan konten tersebut dari *cache*, tanpa perlu menghubungi ALB dan layanan yang berjalan di Amazon EKS. Dengan demikian, jumlah permintaan yang masuk ke bagian belakang sistem menjadi lebih sedikit, sehingga sumber daya komputasi dapat digunakan dengan lebih efektif untuk menangani layanan utama, seperti layanan order, layanan pencocokan, layanan pengemudi, layanan pembayaran, dan layanan notifikasi.

4. Amazon Elastic Kubernetes Service (EKS)

Amazon Elastic Kubernetes Service (EKS) adalah layanan Kubernetes yang disediakan oleh AWS untuk menjalankan, mengelola, dan melakukan penskalaan aplikasi berbasis kontainer. EKS menghilangkan kompleksitas pengelolaan Kubernetes *control plane* karena AWS secara otomatis menangani ketersediaan, skalabilitas, *patching*, dan pemeliharaan komponen inti Kubernetes. Amazon EKS menjalankan *control plane* secara terdistribusi pada beberapa Availability Zone sehingga mampu menyediakan tingkat ketersediaan yang tinggi dan toleransi terhadap kegagalan infrastruktur.

Pada arsitektur ini, Amazon EKS berfungsi sebagai platform utama untuk menjalankan seluruh *microservices* yang menangani proses aplikasi, seperti pemesanan perjalanan, pencarian *driver*, pengelolaan informasi *driver*, pembayaran, dan notifikasi. Setiap *microservice* dibungkus dalam bentuk kontainer dan dijalankan sebagai Kubernetes Pods yang dapat didistribusikan ke beberapa Availability Zone untuk meningkatkan ketersediaan layanan.

M*icroservice* yang dijalankan oleh Amazon EKS:

a. Order Service

Order service bertanggung jawab mengelola seluruh proses pemesanan perjalanan, mulai dari pembuatan order hingga penyelesaian perjalanan. Service ini menyimpan status perjalanan seperti *TripCreated, DriverAssigned, TripStarted,* dan *TripCompleted*.

b. Matching Service

Matching service bertugas mencari dan memilih *driver* yang paling sesuai berdasarkan lokasi, estimasi waktu kedatangan (ETA), status *driver*, dan berbagai parameter lainnya. Service ini merupakan komponen inti yang menentukan kecepatan pengguna mendapatkan *driver*.

c. Driver Service

Driver service mengelola informasi *driver*, status *online*/*offline*, lokasi *real-time*, serta ketersediaan pengemudi. Data yang disimpan oleh layanan ini digunakan oleh Matching Service untuk melakukan proses pencocokan *driver*.

d. Payment Service

Payment Service mengelola seluruh proses pembayaran perjalanan. Ketika order dibuat, service ini dapat menahan saldo terlebih dahulu. Setelah perjalanan selesai dan event *TripCompleted* diterbitkan, Payment Service akan melakukan pembayaran sesuai biaya perjalanan, memperbarui histori transaksi, dan mengirimkan status pembayaran ke layanan lain melalui EventBridge.

e. Notification Service

Notification Service bertanggung jawab mengirimkan *push notification* dalam aplikasi kepada *rider* dan *driver*. Contohnya meliputi notifikasi *driver* ditemukan, perjalanan dimulai, perjalanan selesai, dan pembayaran berhasil.

Alasan penggunaan Amazon EKS pada arsitektur *ride-hailing*:

a. Mendukung Arsitektur *Microservices*

Sistem *ride-hailing* terdiri dari banyak layanan yang memiliki tanggung jawab berbeda, seperti Order Service, Driver Service, Matching Service, Payment Service, dan Notification Service. Amazon EKS menyediakan lingkungan Kubernetes yang dirancang khusus untuk menjalankan aplikasi berbasis *microservices* sehingga setiap layanan dapat dikembangkan dan diskalakan secara independen. Pendekatan ini memungkinkan perubahan pada satu layanan dilakukan tanpa memengaruhi layanan lainnya, sehingga proses pengembangan menjadi lebih fleksibel dan cepat.

b. *High Availability* untuk Layanan Kritis

Sistem *ride-hailing* harus tetap tersedia meskipun terjadi kegagalan pada sebagian infrastruktur. Amazon EKS menyediakan *control plane* yang dikelola AWS dan berjalan pada beberapa Availability Zone. Selain itu, node dan pod dapat didistribusikan ke beberapa AZ sehingga apabila salah satu AZ mengalami gangguan, layanan masih dapat berjalan pada AZ lainnya.

c. Skalabilitas Tinggi untuk Menghadapi Lonjakan Trafik

Jumlah pengguna aplikasi *ride-hailing* dapat berubah secara drastis, misalnya pada jam pulang kerja, akhir pekan, atau saat promosi berlangsung. Amazon EKS memungkinkan aplikasi melakukan penskalaan secara otomatis baik pada tingkat pod maupun node sehingga kapasitas sistem dapat menyesuaikan kebutuhan aktual.

Perbandingan Amazon EKS dengan layanan lainnya:

Dibandingkan dengan Amazon ECS dan Amazon EC2, Amazon EKS dipilih sebagai platform utama untuk menjalankan *microservices* pada arsitektur *ride-hailing* karena menyediakan kemampuan orkestrasi kontainer berbasis Kubernetes yang lebih sesuai untuk aplikasi berskala besar dan kompleks. Menurut CloudForecast, baik ECS maupun EKS merupakan platform orkestrasi container yang dirancang untuk mengotomatisasi proses *deployment*, *scaling*, dan pengelolaan kontainer, sedangkan EC2 hanya menyediakan sumber daya komputasi sehingga seluruh proses orkestrasi harus dikelola secara manual oleh tim operasional.

Pada sistem *ride-hailing* yang terdiri dari berbagai *microservices* seperti Order Service, Matching Service, Driver Service, Payment Service, dan Notification Service, kebutuhan akan otomatisasi *deployment*, *service discovery*, *load balancing*, *self-healing*, dan *autoscaling* menjadi sangat penting sehingga pendekatan berbasis Kubernetes pada Amazon EKS lebih sesuai dibandingkan menjalankan container langsung pada Amazon EC2.

Konsep *autoscaling* pada Amazon EKS:

Pada arsitektur *ride-hailing*, mekanisme *autoscaling* diterapkan pada dua lapisan berbeda, yaitu lapisan aplikasi melalui Horizontal Pod Autoscaler (HPA) dan lapisan infrastruktur melalui Amazon EKS Auto Mode. Kedua

mekanisme tersebut bekerja saling melengkapi untuk memastikan sistem dapat menangani perubahan beban kerja secara otomatis.

HPA digunakan untuk mengatur jumlah Kubernetes Pod yang menjalankan *microservices* berdasarkan metrik tertentu, seperti penggunaan CPU, memori, atau metrik lainnya. Ketika jumlah pengguna meningkat dan beban kerja pada layanan seperti Matching Service atau Driver Service mengalami kenaikan, HPA akan secara otomatis menambahkan replika pod baru agar permintaan pengguna dapat diproses tanpa menyebabkan penurunan performa. Sebaliknya, ketika trafik menurun HPA akan mengurangi jumlah pod untuk mengoptimalkan penggunaan sumber daya.

Di sisi infrastruktur, Amazon EKS Auto Mode bertanggung jawab untuk mengelola kapasitas node yang digunakan oleh *cluster* Kubernetes. Ketika HPA menambahkan pod baru dan kapasitas node yang tersedia tidak lagi mencukupi, EKS Auto Mode akan secara otomatis menyediakan node tambahan yang diperlukan untuk menjalankan pod tersebut. Sebaliknya, ketika jumlah pod berkurang dan terdapat node yang tidak lagi digunakan secara optimal, EKS Auto Mode akan menghapus node tersebut untuk mengurangi biaya operasional.

5. API Gateway WebSocket APIs

Amazon API Gateway WebSocket APIs merupakan layanan manajemen API yang memungkinkan developer untuk membangun komunikasi dua arah (*bidirectional*) secara *real-time* antara aplikasi klien dan *backend* tanpa perlu mengelola infrastruktur server. Layanan ini menyediakan fitur pengelolaan seluruh siklus hidup API, mulai dari tahap pembuatan (*creation*), publikasi (*publish*), pengamanan enkripsi (*protect*), hingga pemantauan *performance metric* (*monitoring*). Melalui kumpulan rute WebSocket, API Gateway bertindak sebagai *traffic distributor* yang terintegrasi secara fleksibel dengan berbagai layanan backend AWS seperti fungsi Lambda, *endpoint* HTTP, maupun layanan AWS lainnya. Berbeda dengan arsitektur HTTP tradisional yang bersifat pasif, koneksi pada WebSocket memungkinkan *backend* untuk mengirim data secara mandiri ke klien secara instan tanpa menunggu adanya *request* dari klien. Berkat kemampuan komunikasi dengan latensi sangat rendah, API Gateway

WebSocket menjadi solusi untuk aplikasi berbasis *real-time* seperti platform obrolan, *multiplayer games*, hingga platform *financial trading*.

Alasan Penggunaan Amazon API Gateway (WebSocket APIs) pada Arsitektur *ride-hailing*:

a. Fasilitator Komunikasi Dua Arah (*Bidirectional*) secara *Real-Time*

   Dalam arsitektur *ride-hailing*, pergerakan koordinat GPS dari pengemudi harus disinkronisasikan ke perangkat penumpang secara instan dalam hitungan milidetik. Amazon API Gateway WebSocket APIs dipilih karena mendukung arsitektur *full-duplex* yang menjaga saluran komunikasi tetap terbuka secara persisten. Karakteristik ini memungkinkan aplikasi pengemudi untuk terus mengalirkan data lokasi terbaru ke sistem. Di saat yang sama, layanan backend dapat secara mandiri langsung mengirim pembaruan posisi tersebut ke peta di aplikasi penumpang secara *real-time*. Mekanisme ini mengeliminasi kebutuhan aplikasi klien untuk melakukan *polling* (permintaan berulang) secara konvensional, sehingga menghasilkan interaksi aplikasi yang lebih responsif, hemat penggunaan kuota internet, serta menghemat konsumsi baterai pada perangkat pengguna.

b. Integrasi Jalur Data dan Tata Kelola Siklus Hidup API

   Amazon API Gateway WebSocket APIs dimanfaatkan sebagai pintu utama yang menyatukan seluruh rute komunikasi dari perangkat luar untuk diintegrasikan secara aman ke arsitektur backend. Setiap pesan yang dikirim oleh klien melalui rute WebSocket tertentu akan diarahkan secara otomatis ke *endpoint* backend yang tepat, seperti fungsi AWS Lambda (untuk penyaringan data koordinat) maupun Amazon Kinesis Data Streams.

Perbandingan API Gateway WebSocket APIs dengan layanan lainnya:

API Gateway WebSocket APIs dan AWS AppSync sama-sama dapat digunakan untuk menyediakan komunikasi real-time antara backend dan aplikasi mobile. Dalam sistem ride-hailing, kedua layanan tersebut memungkinkan server mengirimkan pembaruan status perjalanan maupun lokasi driver secara langsung kepada pengguna tanpa perlu melakukan polling secara berkala. Namun, API Gateway WebSocket lebih sesuai untuk kebutuhan pengiriman data koordinat GPS karena menggunakan protokol WebSocket

standar yang ringan dan dirancang untuk pertukaran pesan dua arah dengan latensi rendah.

Selain itu, API Gateway WebSocket memberikan kontrol yang lebih sederhana terhadap format dan alur pertukaran pesan, sehingga sangat efektif untuk menangani pembaruan lokasi driver yang terjadi secara terus-menerus. Sementara itu, AWS AppSync menggunakan GraphQL Subscription yang menyediakan kemampuan sinkronisasi data yang lebih kaya serta fleksibilitas dalam pengelolaan query dan skema data. Meskipun demikian, fitur tambahan tersebut menambah kompleksitas pemrosesan dan overhead komunikasi yang tidak selalu diperlukan untuk pertukaran data GPS sederhana. Oleh karena itu, untuk sistem dispatch ride-hailing yang berfokus pada efisiensi, latensi rendah, dan pengiriman data lokasi secara real-time, API Gateway WebSocket APIs merupakan pilihan yang lebih tepat dibandingkan AWS AppSync.

6. Amazon Kinesis Data Stream

Amazon Kinesis Data Stream merupakan layanan *data streaming* untuk mengumpulkan dan memproses data berukuran besar dari secara *real-time*. Data dapat berupa log aplikasi, data media sosial, *market data feeds*, dan *web clickstream data*. Selanjutnya data tersebut dapat diproses dan dikirim ke dashboard, digunakan untuk menghasilkan alert, atau mengirimkannya ke layanan AWS lainnya. Kinesis Data Streams memiliki tingkat latensi yang sangat rendah. Karena proses penyerapan ini berjalan secara instan tanpa perlu mengumpulkan data dalam bentuk kelompok (*batching*) terlebih dahulu di sisi server pengirim, data menjadi tersedia untuk diproses hampir seketika dalam hitungan detik.

Alasan penggunaan Amazon Kinesis Data Streams pada arsitektur *ride-hailing*:

a. Menangani Aliran Data GPS Driver Secara *Real-Time*

Aplikasi *ride-hailing* sangat bergantung pada akurasi lokasi *driver* yang memancarkan data koordinat GPS dalam interval yang sangat sering. Amazon Kinesis Data Streams digunakan sebagai gerbang utama untuk menyerap seluruh aliran data lokasi yang masuk dari ponsel *driver* di seluruh Indonesia secara *real-time*. Kinesis memastikan tingkat ketahanan data yang tinggi (*durability*) serta sifat elastis (*elasticity*). *Elasticity* memungkinkan untuk melakukan *scale up*

atau *scale down* sesuai fluktuasi *traffic*. Dengan pemrosesan *real-time* yang instan, sistem dapat mencegah hilangnya data koordinat penting apabila terjadi kegagalan, sehingga posisi geografis setiap pengemudi untuk kebutuhan pelacakan perjalanan selalu akurat.

b. Mendukung Concurrent Consumption

Salah satu keunggulan utama Kinesis Data Streams adalah kemampuannya untuk mengizinkan beberapa aplikasi konsumen membaca data dari satu aliran *stream* yang sama secara bersamaan (*concurrently*) dan independen. Dalam *ride-hailing*, aliran data GPS yang masuk dapat dicabangkan secara paralel untuk kebutuhan yang berbeda tanpa saling mengganggu.

Perbandingan Amazon Kinesis Data Stream dengan layanan lainnya:

Amazon Kinesis Data Streams dan Amazon MSK (Apache Kafka) sama-sama dapat digunakan untuk membangun pipeline streaming data *real-time* yang mampu menangani aliran data GPS driver secara kontinu. Kedua layanan mendukung proses *ingest*, penyimpanan sementara, dan distribusi data ke berbagai layanan pemrosesan maupun analitik. Namun, Kinesis Data Streams lebih cocok digunakan ketika seluruh arsitektur sistem berada di lingkungan AWS karena memiliki integrasi yang erat dengan layanan AWS lainnya, seperti AWS Lambda dan Amazon S3. Integrasi tersebut memungkinkan proses pengembangan dan implementasi pipeline streaming dilakukan dengan lebih cepat dan sederhana.

Alasan utama memilih Kinesis Data Streams adalah rendahnya kompleksitas operasional dibandingkan Amazon MSK. Dengan mode On-Demand, kapasitas Kinesis dapat menyesuaikan secara otomatis terhadap perubahan volume data tanpa perlu melakukan konfigurasi broker atau pengelolaan kluster secara langsung. Sebaliknya, Amazon MSK dirancang untuk memberikan kompatibilitas penuh dengan ekosistem Apache Kafka, sehingga menawarkan fleksibilitas yang lebih tinggi tetapi juga membutuhkan pengelolaan dan konfigurasi yang lebih kompleks, terutama pada deployment berbasis provisioned cluster. Oleh karena itu, untuk kebutuhan pemantauan lokasi driver secara real-time dengan fokus pada kemudahan implementasi, efisiensi operasional, dan integrasi yang kuat dengan layanan AWS, Kinesis Data Streams menjadi pilihan yang lebih sesuai.

7. AWS Lambda

AWS Lambda merupakan layanan komputasi *serverless* yang dapat memproses kode program secara otomatis tanpa perlu mengelola, menyediakan kapasitas (*provisioning*), dan memelihara sistem operasi pada server. Lambda beroperasi dengan ketersediaan tinggi (*high-availability*) dan menerapkan sistem pembayaran *pay-per-use*, di mana pengguna hanya membayar layanan yang digunakan. Kode program yang diatur di dalam fungsi Lambda akan langsung dieksekusi ketika dipicu oleh kejadian tertentu lewat pengiriman data berformat JSON. Selain itu, Lambda memiliki kemampuan manajemen skalabilitas otomatis dan *concurrency controls* yang sangat presisi untuk menjaga responsivitas aplikasi ketika terjadi lonjakan beban kerja yang masif.

Alasan penggunaan AWS Lambda pada arsitektur *ride-hailing*:

a. *Stream Processing* Koordinat GPS Efisien

Dalam arsitektur *ride-hailing*, AWS Lambda digunakan sebagai komponen pemroses utama yang terhubung langsung dengan Amazon Kinesis Data Streams (*event source mapping*). Lambda mengambil serta mengolah aliran data koordinat lokasi *driver* yang masuk secara *real-time*. Sifat *serverless* Lambda membuat sistem tidak perlu mengalokasikan server komputasi yang menyala terus-menerus untuk menunggu data masuk. Komputasi Lambda hanya akan aktif dan memakan biaya ketika ada data GPS yang dialirkan, sehingga proses analisis pergerakan kendaraan dan pemetaan koordinat dapat berjalan secara ringan, instan, dan ekonomis.

b. Menjadi *Ingestion Mediator* untuk Mencegah Kebocoran *Workload*

AWS Lambda dimanfaatkan sebagai jembatan sebelum data dari Kinesis diteruskan menuju database dan *microservices* di dalam Amazon EKS. Dalam VPC, fungsi Lambda diposisikan untuk melakukan penyaringan data, transformasi format, serta pengelompokan (*batching*) data koordinat *driver*. Dengan melakukan proses tersebut di Lambda, data yang dikirimkan ke Amazon Aurora PostgreSQL, Amazon DynamoDB, dan Amazon ElastiCache merupakan data penting yang sudah bersih. Mekanisme ini memangkas beban kerja komputasi pada Amazon EKS serta mengamankan

database transaksional dari risiko *overload* akibat jutaan permintaan penulisan data langsung per detik.

Perbandingan Amazon Lambda dengan layanan lainnya:

AWS Lambda dan Amazon EC2 sama-sama dapat digunakan sebagai lapisan komputasi untuk memproses data GPS yang diterima dari layanan streaming. Namun, AWS Lambda dipilih karena menerapkan arsitektur serverless yang memungkinkan fungsi dijalankan secara otomatis sebagai respons terhadap event tanpa perlu menyediakan atau mengelola server secara langsung. Integrasi Lambda dengan layanan streaming AWS juga memudahkan proses pemrosesan data secara real-time, sehingga pengembang dapat lebih fokus pada implementasi logika bisnis dibandingkan pengelolaan infrastruktur.

Selain itu, AWS Lambda menggunakan model pembayaran pay-per-use, yaitu biaya dihitung berdasarkan jumlah permintaan dan durasi eksekusi fungsi. Dengan pendekatan ini, biaya komputasi hanya muncul ketika terdapat data GPS yang diproses. Sebaliknya, penggunaan Amazon EC2 mengharuskan instance server tetap aktif untuk menerima dan memproses data, sehingga biaya tetap dikenakan selama server berjalan meskipun tingkat aktivitas sistem sedang rendah. Oleh karena itu, untuk aplikasi pelacakan driver yang memiliki pola beban kerja dinamis dan membutuhkan efisiensi biaya serta skalabilitas otomatis, AWS Lambda menjadi pilihan yang lebih sesuai dibandingkan Amazon EC2.

8. AWS Step Functions

AWS Step Functions adalah layanan orkestrasi alur kerja (*visual workflow orchestrator*) berbasis *serverless* yang memungkinkan developer untuk menggabungkan berbagai layanan AWS, aktivitas kustom, atau *microservices* menjadi sebuah alur proses bisnis yang teratur. layanan orkestrasi alur kerja (*visual workflow orchestrator*) berbasis *serverless* yang memungkinkan developer untuk menggabungkan berbagai layanan AWS, aktivitas kustom, atau *microservices* menjadi sebuah alur proses bisnis yang teratur.

Proses aplikasi *ride-hailing* memiliki sifat yang asinkron, memiliki banyak tahapan, dan sangat bergantung pada kondisi *real-time*. Jika menggunakan pendekatan koordinasi antar layanan secara langsung, akan

membuat sistem menjadi rumit dan rapuh (*tightly coupled*). Pada *design* arsitektur *ride-hailing*, AWS Step Function diletakkan di dalam VPC untuk mempermudah gambaran *workflow*.

Alasan penggunaan AWS Step Functions pada arsitektur *ride-hailing*:

a. Menyediakan *Error Handling* dan *Retry* Otomatis

   Dalam lingkungan terdistribusi, proses pembaruan data dapat mengalami kegagalan sementara akibat timeout, gangguan jaringan, atau kendala layanan lainnya. AWS Step Functions menyediakan fitur *Retry* dan *Catch* yang memungkinkan *workflow* mencoba kembali langkah yang gagal secara otomatis tanpa memerlukan implementasi logika tambahan pada aplikasi. Kemampuan ini meningkatkan keandalan sistem dan membantu memastikan bahwa proses penyimpanan data tetap dapat diselesaikan meskipun terjadi gangguan sementara.

b. Meningkatkan Observabilitas dan Kemudahan Monitoring *Workflow*

   AWS Step Functions menyediakan visualisasi *workflow* yang memungkinkan tim operasional memantau setiap tahapan proses secara *real-time*. Pada sistem *ride-hailing*, kemampuan ini membantu tim dalam melakukan investigasi ketika terjadi kegagalan pada proses pembaruan data transaksi, histori perjalanan, maupun data analitik.

Perbandingan Amazon Step Functions dengan layanan lainnya:

AWS Step Functions dan Apache Airflow (termasuk Amazon MWAA) sama-sama berfungsi sebagai workflow orchestrator yang mengatur urutan dan dependensi antarproses dalam suatu sistem. Pada aplikasi ride-hailing, layanan ini dapat digunakan untuk mengelola alur transaksi seperti penerimaan pesanan, pencocokan driver, konfirmasi perjalanan, hingga penyelesaian pembayaran. Namun, AWS Step Functions lebih sesuai karena dirancang untuk orkestrasi aplikasi berbasis event dan memiliki integrasi native dengan berbagai layanan AWS, seperti AWS Lambda, Amazon DynamoDB, dan Amazon SQS, sehingga implementasi workflow dapat dilakukan dengan lebih sederhana dan efisien.

Selain itu, AWS Step Functions mampu menangani workflow transaksi secara real-time dengan latensi yang rendah dan dukungan monitoring yang terintegrasi. Sebaliknya, Apache Airflow lebih banyak digunakan pada

lingkungan data engineering untuk mengelola proses ETL, data warehouse, dan batch processing berskala besar yang umumnya tidak memiliki tuntutan respons instan. Oleh karena itu, untuk sistem dispatch ride-hailing yang membutuhkan pengambilan keputusan dan koordinasi layanan secara cepat dalam setiap transaksi, AWS Step Functions merupakan pilihan yang lebih tepat dibandingkan Apache Airflow atau Amazon MWAA.

9. Amazon Aurora

Amazon Aurora merupakan layanan database relasional terkelola penuh (*fully managed*) yang dikembangkan oleh AWS dengan kompatibilitas penuh terhadap mesin database *open source* populer, yaitu MySQL dan PostgreSQL. Sebagai bagian dari ekosistem Amazon Relational Database Service (Amazon RDS), Aurora menggabungkan kecepatan, keandalan, dan performa tinggi khas database komersial kelas atas dengan kesederhanaan serta efisiensi biaya dari database open-source.

Aurora mampu melakukan *autoscaling* hingga ukuran maksimal mencapai 256 tebibytes (TiB). Tanpa memerlukan perubahan pada kode program atau aplikasi yang sudah ada, Aurora mampu memberikan performa dengan *throughput* hingga lima kali lipat dibanding MySQL standar dan hingga tiga kali lipat dibanding PostgreSQL standar.

Alasan penggunaan Amazon Aurora pada arsitektur *ride-hailing*:

a. Penyimpanan Skala Besar Berkinerja Tinggi untuk Data Transaksional

Dalam operasional *ride-hailing*, sistem selalu dibanjiri oleh data transaksional yang terjadi setiap detik, mulai dari proses pemesanan perjalanan, pencatatan tarif, transaksi pembayaran, hingga riwayat perjalanan pengguna. Amazon Aurora dipilih sebagai repositori utama karena arsitektur penyimpanan yang canggih dan mampu memberikan *throughput* performa hingga berkali-kali lipat dibanding mesin database standar. Dengan kemampuan volume klaster yang dapat tumbuh otomatis hingga 256 TiB, sistem tidak akan mengalami kendala kehabisan ruang penyimpanan akibat lonjakan data transaksi yang tinggi. Hal ini memastikan bahwa seluruh pemrosesan data transaksional berjalan dengan latensi rendah, instan, dan memiliki keandalan setara *database* komersial namun dengan efisiensi biaya yang tetap terjaga.

b. Otomatisasi Klasterisasi dan Replikasi Tinggi untuk Menjamin Ketersediaan Layanan

Aplikasi *ride-hailing* menuntut ketersediaan sistem yang sangat tinggi (*high availability*) tanpa adanya toleransi terhadap *downtime* yang dapat merugikan jutaan pengguna. Amazon Aurora mereduksi kompleksitas administrasi data dengan melakukan otomatisasi penuh pada proses klasterisasi, replikasi data, serta deteksi kerusakan (*failure detection and repair*). Melalui integrasi dengan Amazon RDS, Aurora mengelola sinkronisasi antar server di dalam klaster secara otomatis. Jika terjadi kegagalan pada satu *instance database*, mekanisme pemulihan akan langsung berjalan secara instan untuk mengalihkan beban kerja ke *instance* replika tanpa mengganggu jalannya aplikasi, sehingga operasional bisnis *ride-hailing* dapat terus berjalan mulus dan andal selama 24 jam.

Perbandingan Amazon Aurora dengan layanan lainnya:

Amazon Aurora dan Amazon RDS for PostgreSQL sama-sama menyediakan layanan database relasional yang mendukung transaksi ACID untuk menyimpan data penting, seperti informasi pembayaran, pemesanan perjalanan, dan riwayat transaksi pengguna. Namun, Amazon Aurora dipilih karena merupakan database yang dirancang secara khusus untuk lingkungan cloud dengan arsitektur penyimpanan yang terpisah dari lapisan komputasi. Pendekatan ini memungkinkan Aurora memberikan performa yang lebih tinggi serta skalabilitas yang lebih baik dibandingkan PostgreSQL standar yang dijalankan melalui Amazon RDS.

Selain aspek performa, Amazon Aurora menawarkan tingkat ketersediaan dan ketahanan data yang lebih tinggi. Aurora secara otomatis mereplikasi data ke tiga Availability Zone dalam satu region AWS dengan total enam salinan data yang tersebar secara terdistribusi. Mekanisme ini membantu menjaga kontinuitas layanan apabila terjadi kegagalan pada server maupun Availability Zone tertentu. Oleh karena itu, untuk sistem ride-hailing yang menangani transaksi pembayaran dan data operasional yang bersifat kritis, Amazon Aurora menjadi pilihan yang lebih sesuai karena mampu memberikan kombinasi performa, keandalan, dan ketersediaan yang lebih baik dibandingkan Amazon RDS PostgreSQL standar.

10. Amazon DynamoDB

Amazon DynamoDB merupakan layanan database NoSQL terdistribusi berbasis *serverless* yang dikelola penuh (*fully managed*) oleh AWS untuk menghasilkan performa latensi *single-digit millisecond*. DynamoDB dirancang untuk mengatasi kompleksitas operasional serta keterbatasan skalabilitas yang umum ditemukan pada database relasional konvensional. DynamoDB mendukung model data *key value* dan dokumen yang membuat struktur data menjadi lebih fleksibel. Berbeda dengan database relasional, DynamoDB tidak menyediakan operasi 'JOIN' demi menghindari penurunan performa. Sistem ini justru mengutamakan denormalisasi data agar setiap *query* dapat dieksekusi dengan kecepatan maksimal. Layanan ini menghilangkan beban kerja infrastruktur seperti penyediaan server, pembaruan sistem (*patching*), dan *zero downtime maintenance*. Dengan mode kapasitas *on demand*, DynamoDB menerapkan sistem pembayaran berbasis *pay-as-you-go* yang menaikkan atau menurunkan skala secara otomatis hingga menyentuh titik nol saat tidak ada trafik, tanpa memicu adanya *cold start*. Untuk kebutuhan tingkat perusahaan, DynamoDB menjamin ketersediaan tinggi melalui replikasi data otomatis di tiga Availability Zone, perlindungan enkripsi data (*encryption at rest*), serta pemenuhan kepatuhan terhadap transaksi ACID yang konsisten.

Alasan penggunaan Amazon DynamoDB pada arsitektur *ride-hailing*:

a. Manajemen Data Sesi Aktif dan Pelacakan Status *Driver*

Dalam ekosistem *ride-hailing*, sistem harus mencatat dan memperbarui status dinamis dari ratusan ribu pengemudi (*driver*) setiap detik, seperti status aktif (*online/offline*), kesiapan menerima order, hingga ID sesi perjalanan yang sedang berjalan. Amazon DynamoDB dipilih karena kemampuannya dalam memberikan performa pembacaan dan penulisan data yang konsisten dalam hitungan milidetik, bahkan saat menangani lonjakan trafik masif hingga jutaan permintaan per detik. Kemampuan skalabilitas horizontalnya menjamin bahwa pencarian status *driver* terdekat dapat dieksekusi secara instan dan responsif demi memberikan pengalaman pengguna yang mulus tanpa terpengaruh oleh jumlah pengguna aplikasi yang melonjak tinggi.

b. Sinkronisasi Arsitektur Berbasis Peristiwa (*Event-Driven*) Lewat Fitur Change Data Capture (CDC)

Setiap perubahan status operasional dalam layanan *ride-hailing* (misalnya ketika status pemesanan berubah dari "mencari driver" menjadi "driver dalam perjalanan") harus langsung memicu serangkaian aksi di layanan lain secara *near-real time*. DynamoDB menyediakan solusi ideal melalui integrasi *native* fitur *Change Data Capture* (CDC), seperti DynamoDB Streams yang terhubung langsung dengan AWS Lambda. Setiap kali terjadi pembuatan atau pembaruan item pada tabel, runtutan perubahan tersebut dialirkan dalam urutan waktu yang presisi untuk menjalankan fungsi komputasi berikutnya secara otomatis. Mekanisme pengiriman pesan otomatis ini mempermudah pengembangan arsitektur mikro (*microservices*) yang reaktif, aman, dan efisien tanpa membebani performa basis data utama.

11. Amazon ElastiCache

Amazon ElastiCache merupakan layanan penyimpanan data di dalam memori (*in-memory data store*) terkelola penuh dari AWS yang berfungsi sebagai repositori data sementara (*cache*). Layanan ini dirancang untuk menyederhanakan proses penyiapan, pengoperasian, serta penskalaan infrastruktur *cache* terdistribusi dengan mendukung tiga mesin (*engine*) utama, yaitu Valkey, Redis OSS, dan Memcached. Developer dapat mengoperasikan ElastiCache dalam dua format sesuai kebutuhan kendali arsitekturnya. Format pertama adalah *Serverless*, di mana sistem secara otomatis memantau serta menaikkan kapasitas memori, komputasi, dan *bandwidth* jaringan dalam hitungan menit tanpa perlu melakukan konfigurasi node ataupun pengelolaan perangkat keras secara manual. Format kedua adalah *Node-based Cluster*, yang memberikan kontrol penuh bagi pengembang untuk menentukan tipe instance, jumlah node, tata letak Multi-AZ untuk ketersediaan tinggi, hingga fitur persistensi data lewat log transaksional.

Alasan penggunaan Amazon ElastiCache pada arsitektur *ride-hailing*:

a. Mengurangi Latensi Pembacaan Data Statis dan Informasi Sesi yang Sering Diakses

Dalam operasional aplikasi *ride-hailing*, ada banyak data yang sifatnya jarang berubah namun terus-menerus diakses oleh jutaan pengguna setiap detiknya, seperti data tarif per kilometer, koordinat *driver* terakhir untuk dicocokan dengan penumpang, daftar promo aktif, profil dasar pengguna, hingga pemetaan area layanan. Amazon ElastiCache digunakan untuk menyimpan data-data tersebut langsung di dalam memori RAM (*in-memory*). Dengan memotong jalur pencarian data yang biasanya harus mengambil ke database utama, ElastiCache mampu merespons permintaan aplikasi dalam waktu hitungan mikrodetik. Hasilnya, aplikasi tetap terasa sangat responsif dan ringan saat memuat informasi, sekaligus menghemat beban kerja komputasi pada database transaksional utama.

b. Manajemen Beban Kerja Dinamis saat Terjadi Lonjakan Trafik

Aplikasi *ride-hailing* sangat rentan terhadap lonjakan trafik pada waktu-waktu tertentu, misalnya saat jam pulang kantor, kondisi cuaca hujan, atau festival besar. Dengan memanfaatkan opsi *ElastiCache Serverless*, sistem penyimpanan sementara ini dapat langsung menaikkan skala kapasitasnya secara otomatis demi mengimbangi lonjakan permintaan pencarian armada atau penghitungan tarif yang naik drastis. Kemampuan adaptasi instan ini memastikan arsitektur aplikasi tidak mengalami penurunan performa akibat kehabisan memori, sehingga kenyamanan pengguna dalam memesan perjalanan tetap terjaga di segala situasi pasar.

Perbandingan Amazon ElastiCache dengan layanan lainnya:

Amazon ElastiCache dan Amazon MemoryDB for Redis sama-sama menyediakan penyimpanan data berbasis memori dengan performa tinggi yang dapat digunakan untuk mengelola data lokasi driver, informasi geospasial, serta cache yang membutuhkan akses dalam waktu sangat singkat. Pada sistem ride-hailing, data lokasi driver bersifat sangat dinamis dan terus diperbarui, sehingga membutuhkan layanan yang mampu memberikan latensi baca dan tulis yang rendah. Dalam konteks ini, Amazon ElastiCache menjadi pilihan

yang sesuai karena dioptimalkan untuk kebutuhan caching dan pemrosesan data real-time dengan performa yang sangat tinggi.

Selain itu, Amazon ElastiCache memiliki biaya operasional yang lebih rendah dibandingkan Amazon MemoryDB. MemoryDB dirancang untuk menyediakan durabilitas data yang tinggi melalui mekanisme persistensi dan replikasi lintas Availability Zone, sehingga dapat digunakan sebagai database utama berbasis Redis. Namun, kemampuan tersebut menambah kompleksitas proses penulisan data dan umumnya menghasilkan latensi tulis yang lebih tinggi dibandingkan ElastiCache. Oleh karena itu, untuk kebutuhan penyimpanan data lokasi driver dan caching yang tidak memerlukan persistensi permanen, Amazon ElastiCache for Redis merupakan pilihan yang lebih efisien dari sisi performa maupun biaya.

12. Amazon S3

Amazon Simple Storage Service (Amazon S3) merupakan layanan penyimpanan berbasis objek (*object storage*) terkelola penuh dari AWS. Amazon S3 menyimpan data sebagai objek di dalam kontainer yang disebut *bucket*, di mana setiap objek diidentifikasi secara unik melalui kombinasi nama *bucket*, kunci (*object key*), dan ID versi. Layanan ini menyediakan berbagai pilihan kelas penyimpanan yang dapat dioptimalkan sesuai pola akses data, mulai dari S3 Standard untuk data produksi yang sering diakses, hingga lini S3 Glacier untuk pengarsipan jangka panjang. Guna mempermudah tata kelola data dalam jumlah besar, Amazon S3 dilengkapi dengan fitur manajemen siklus hidup objek (*S3 Lifecycle*), replikasi otomatis antar-wilayah (*S3 Replication*), serta kontrol akses granular berbasis IAM Policy, *Bucket Policy*, dan *S3 Block Public Access* untuk memastikan keamanan data tetap terjaga.

Alasan penggunaan Amazon S3 pada arsitektur *ride-hailing*:

a. *Data Lake* dan Penyimpanan Dokumen Kepatuhan Pengguna yang Scalable

Operasional bisnis *ride-hailing* menghasilkan volume data non-transaksional yang sangat masif setiap harinya, seperti dokumen pendaftaran mitra pengemudi (foto KTP, SIM, STNK), berkas riwayat perjalanan format teks, hingga data mentah log GPS untuk analitik *big data*. Amazon S3 dipilih sebagai fondasi *Data Lake* karena

kemampuan skalabilitasnya yang tidak terbatas untuk menampung data dalam ukuran petabyte sekalipun tanpa memicu penurunan performa. Integrasinya yang luas dengan layanan analitik seperti Apache Spark atau Amazon EMR mempermudah tim data science untuk mengekstrak wawasan dari data historis tersebut secara ekonomis serta memanfaatkan fitur *S3 Lifecycle* untuk otomatis memindahkan dokumen lama ke kelas penyimpanan yang lebih murah seperti S3 Glacier.

b. Pengamanan Aset Digital dan Isolasi Hak Akses Data Sensitif

Sebagai aplikasi publik yang menangani data identitas sensitif milik jutaan pengguna dan pengemudi, arsitektur *ride-hailing* wajib menerapkan proteksi perimeter keamanan yang ketat demi mencegah kebocoran data. Dengan memanfaatkan Amazon S3, seluruh *bucket* penyimpanan secara bawaan dikonfigurasi privat lewat fitur *S3 Block Public Access*. Otentikasi dan otorisasi akses terhadap dokumen atau data riwayat perjalanan dikendalikan menggunakan *AWS Identity and Access Management* (IAM) serta *Bucket Policies*. Mekanisme ini memastikan bahwa aset digital penting hanya dapat diakses oleh mikroservis internal yang sah di dalam arsitektur cloud, sekaligus melindungi data dari risiko modifikasi atau penghapusan yang tidak disengaja oleh pihak luar.

13. AWS Secrets Manager

AWS Secrets Manager layanan yang digunakan untuk mengelola, mengambil, dan menyimpan informasi sensitif (*secrets*) seperti kredensial database, API keys, token autentikasi, dan password aplikasi secara aman. Secrets Manager melakukan enkripsi seluruh informasi sensitif menggunakan AWS Key Management Service (AWS KMS) dan mendukung rotasi kredensial secara otomatis. Selain itu, AWS Secrets Manager terintegrasi dengan AWS IAM untuk mengontrol siapa atau layanan yang dapat mengakses informasi tertentu.

Alasan penggunaan AWS Secrets Manager pada arsitektur *ride-hailing*:

a. Pengelolaan Kredensial Aplikasi yang Aman

Pada sistem *ride-hailing*, berbagai layanan butuh informasi sensitif seperti kredensial database Amazon Aurora, API key layanan

pembayaran, token notifikasi, maupun kredensial integrasi dengan layanan pihak ketiga. Menyimpan informasi tersebut secara langsung di dalam kode aplikasi atau file konfigurasi kontainer bisa berisiko menimbulkan kebocoran data. AWS Secrets Manager dipilih karena menyediakan penyimpanan terenkripsi yang terintegrasi dengan AWS Key Management Service, sehingga kredensial dapat dikelola secara terpusat dan diakses secara aman oleh *microservices* yang berjalan pada Amazon EKS.

b. Rotasi Kredensial Otomatis dan Pengurangan Risiko Keamanan

Dalam lingkungan produksi yang besar, penggunaan kredensial yang sama dalam waktu lama dapat meningkatkan risiko penyalahgunaan jika kredensial tersebut berhasil diperoleh oleh pihak yang tidak berwenang. AWS Secrets Manager menyediakan mekanisme *secret rotation* otomatis yang memungkinkan password database atau token autentikasi diperbarui secara berkala tanpa mengganggu operasional aplikasi. Hal ini membantu meningkatkan keamanan sistem *ride-hailing* dan mengurangi beban operasional tim DevOps dalam melakukan pengelolaan kredensial secara manual.

Perbandingan Amazon Secrets Manager dengan layanan lainnya:

AWS Secrets Manager dan AWS Systems Manager Parameter Store sama-sama dapat digunakan untuk menyimpan informasi sensitif, seperti kredensial database, API key, token autentikasi, dan konfigurasi aplikasi yang bersifat rahasia. Kedua layanan juga mendukung integrasi dengan AWS Identity and Access Management (IAM) serta enkripsi menggunakan AWS Key Management Service (KMS). Namun, AWS Secrets Manager dirancang secara khusus untuk pengelolaan secret pada lingkungan produksi yang membutuhkan tingkat keamanan dan otomatisasi yang lebih tinggi.

Keunggulan utama AWS Secrets Manager adalah dukungan automatic secret rotation yang terintegrasi secara native dengan layanan database AWS, termasuk Amazon Aurora dan Amazon RDS. Fitur ini memungkinkan kredensial database diperbarui secara berkala secara otomatis tanpa mengganggu operasional aplikasi, sehingga membantu meningkatkan keamanan sistem dan mengurangi risiko penggunaan kredensial yang sudah lama. Sementara itu, AWS Systems Manager Parameter Store lebih cocok

digunakan untuk penyimpanan konfigurasi aplikasi dan parameter sistem yang relatif statis, karena implementasi rotasi kredensial memerlukan mekanisme tambahan di luar fitur bawaan layanan. Oleh karena itu, untuk penyimpanan database credentials dan API key pada sistem ride-hailing yang bersifat kritis, AWS Secrets Manager merupakan pilihan yang lebih sesuai.

14. AWS Identity and Access Management (IAM)

AWS Identity and Access Management (IAM) adalah layanan AWS yang digunakan untuk mengelola autentikasi dan otorisasi sumber daya di lingkungan AWS. AWS IAM memungkinkan administrator menentukan siapa yang dapat mengakses sumber daya AWS dan tindakan apa saja yang dapat dilakukan melalui mekanisme *user*, *role*, dan *policy*. Selain itu, AWS IAN juga terintegrasi dengan berbagai layanan AWS, seperti AWS Secrets Manager dan Amazon EventBridge untuk mengontrol hak akses terdapat sumber daya yang digunakan aplikasi.

Alasan penggunaan AWS IAM dalam arsitektur ride-hailing:

a. Penerapan Prinsip *Least Privilege* pada *Microservices*

Arsitektur *ride-hailing* terdiri dari beberapa microservice seperti Order Service, Matching Service, Driver Service, Payment Service, dan Notification Service. AWS IAM digunakan untuk menerapkan prinsip *least privilege* dengan memberikan hak akses minimum yang diperlukan oleh setiap layanan. Dengan prinsip ini, AWS IAM membantu meminimalkan dampak jika salah satu layanan mengalami masalah keamanan.

b. Pengelolaan Hak Akses Terpusat dan Aman

AWS IAM memungkinkan seluruh hak akses terhadap layanan AWS dikelola secara terpusat melalui mekanisme *role* dan *policy*. Dalam sistem *ride-hailing*, berbagai komponen layanan seperti Amazon EKS, AWS Lambda, Amazon EventBridge, dan AWS Secrets Manager dapat menggunakan IAM Role yang berbeda sesuai kebutuhan masing-masing. Pengelolaan akses yang terpusat ini memudahkan proses audit, meningkatkan keamanan sistem, serta mengurangi risiko kesalahan konfigurasi akses yang dapat menyebabkan kebocoran data atau penyalahgunaan sumber daya AWS.

15. Amazon CloudWatch

Amazon CloudWatch adalah layanan yang digunakan untuk memantau sumber daya AWS dan aplikasi yang dijalankan secara *real-time*. Amazon CloudWatch mengumpulkan, menyimpan, dan menganalisis metrik, log, dan data operasional dari aplikasi dan infrastruktur AWS. Amazon CloudWatch menyediakan visibilitas terhadap performa aplikasi, kesehatan sistem, penggunaan sumber daya, dan kondisi operasional secara keseluruhan. Layanan ini juga mendukung pengumpulan log serta pemantauan aplikasi berbasis kontainer melalui fitur seperti *Container Insights*.

Alasan penggunaan Amazon CloudWatch pada arsitektur ride-hailing:

a. Monitoring Kesehatan dan Performa Sistem Secara *Real-Time*

   Layanan *ride-hailing* harus mampu melayani ribuan hingga jutaan pengguna secara bersamaan dengan tingkat ketersediaan yang tinggi. Amazon CloudWatch digunakan untuk memantau metrik operasional seperti penggunaan CPU dan memori pada Amazon EKS, jumlah permintaan yang diterima oleh ALB, performa database Aurora, serta *throughput* layanan streaming seperti Amazon Kinesis. Dengan pemantauan secara *real-time*, tim operasional dapat mendeteksi gangguan atau penurunan performa lebih cepat sebelum berdampak pada pengguna.

b. *Centralized Logging* untuk *Troubleshooting* dan *Observability*

   Dalam arsitektur *microservices*, proses bisnis seperti pemesanan kendaraan, pencocokan *driver*, pembayaran, dan pengiriman notifikasi melibatkan banyak layanan yang saling berinteraksi. Amazon CloudWatch memungkinkan seluruh log aplikasi dikumpulkan dalam satu lokasi terpusat sehingga memudahkan proses investigasi ketika terjadi kegagalan transaksi atau error pada aplikasi.

c. Otomatisasi Respons Terhadap Gangguan Sistem

   Amazon CloudWatch menyediakan fitur alarm yang dapat memantau kondisi tertentu dan memicu tindakan otomatis ketika mencapai suatu batas tertentu. Ketika penggunaan CPU pada *cluster* Amazon EKS meningkat signifikan, CloudWatch dapat mengirimkan notifikasi atau memicu *workflow* otomatis untuk menangani kondisi

tersebut. Mekanisme ini membantu menjaga stabilitas dan ketersediaan layanan secara berkelanjutan.

16. AWS CloudTrail

AWS CloudTrail adalah layanan audit dan pencatatan aktivitas yang merekam seluruh aktivitas API dan perubahan konfigurasi yang terjadi pada akun AWS. AWS CloudTrail menyediakan riwayat aktivitas yang dapat digunakan untuk mengetahui siapa yang melakukan suatu tindakan, kapan, serta dari mana tindakan tersebut berasal. Log CloudTrail dapat disimpan ke Amazon S3 dan dianalisis untuk kebutuhan audit, keamanan, dan kepatuhan (*compliance*).

Alasan penggunaan AWS CloudTrail pada arsitektur *ride-hailing*:

a. Audit dan Pelacakan Aktivitas Operasional

Sistem *ride-hailing* mengelola berbagai data penting, termasuk data pengguna, transaksi pembayaran, dan konfigurasi infrastruktur cloud. AWS CloudTrail digunakan untuk merekam seluruh aktivitas API yang terjadi pada akun AWS, termasuk perubahan konfigurasi layanan, akses ke Amazon Secrets Manager, maupun perubahan kebijakan AWS IAM. Dengan adanya pencatatan aktivitas ini, organisasi dapat mengetahui siapa yang melakukan suatu tindakan, kapan tindakan tersebut dilakukan, dan sumber akses yang digunakan.

b. Mendukung Investigasi Keamanan dan Kepatuhan

Jika terjadi masalah keamanan atau perubahan konfigurasi yang tidak diinginkan, AWS CloudTrail menyediakan jejak audit (*audit trail*) yang dapat digunakan untuk melakukan investigasi secara mendetail. Informasi ini dapat digunakan untuk mengidentifikasi sumber masalah, mengevaluasi dampak insiden, serta memenuhi kebutuhan audit dan kepatuhan terhadap standar keamanan perusahaan maupun regulasi yang berlaku.

17. Amazon EventBridge

Amazon EventBridge adalah layanan *serverless event bus* yang menungkinkan berbagai layanan berkomunikasi menggunakan mekanisme *event*. Amazon EventBridge menerima *event* dari berbagai sumber kemudian meneruskan event tersebut ke target yang sesuai berdasarkan aturan yang telah

ditentukan. Amazon EventBridge mendukung arsitektur *event-driven* yang terdistribusi dan *loosely coupled*.

Alasan penggunaan Amazon EventBridge pada arsitektur *ride-hailing*:

a. Mendukung Arsitektur Event-Driven

   Operasional layanan *ride-hailing* menghasilkan berbagai *event* yang terjadi secara terus-menerus seperti pembuatan order baru, pemilihan *driver* yang cocok, perubahan status perjalanan, penyelesaian pembayaran, dan pengiriman notifikasi. Amazon EventBridge digunakan sebagai *event bus* tanpa harus memiliki ketergantungan langsung satu sama lain. Hal ini dapat meningkatkan fleksibilitas sistem dan mempermudah pengembangan layanan baru.

b. Orkestrasi Workflow Antar Layanan Secara Asinkron

   Dalam sistem *ride-hailing*, satu *event* dapat memicu serangkaian proses yang melibatkan beberapa layanan secara bersamaan. Ketika pengguna berhasil membuat pesanan (*TripCreated*), EventBridge akan mendistribusikan *event* tersebut ke berbagai layanan terkait. Matching Service akan memulai proses pencarian pengemudi terdekat, Notification Service akan mengirimkan notifikasi kepada pengemudi dan pengguna, sedangkan Payment Service dapat melakukan proses menahan saldo pada metode pembayaran pengguna untuk memastikan ketersediaan dana. Setelah perjalanan selesai dan Order Service menerbitkan *event TripCompleted*, EventBridge akan meneruskan *event* tersebut ke beberapa layanan secara asinkron. Payment Service akan melakukan penyelesaian transaksi sesuai biaya perjalanan, Order Service akan memperbarui status perjalanan menjadi selesai, dan Notification Service akan mengirimkan ringkasan perjalanan kepada pengguna. Dengan Amazon EventBridge, seluruh *workflow* tersebut dapat dijalankan secara otomatis tanpa ketergantungan langsung antar layanan, sehingga mengurangi kompleksitas integrasi dan meningkatkan skalabilitas sistem.

c. Otomatisasi Operasional dan Integrasi Layanan AWS

   Amazon EventBridge juga dapat digunakan untuk merespons berbagai *event* yang berasal dari layanan AWS lainnya seperti

perubahan status sumber daya, alarm CloudWatch, atau aktivitas tertentu dalam sistem.

D. Alur Arsitektur

Sistem ini dirancang menggunakan arsitektur *Single-Region Multi-AZ* untuk memisahkan beban kerja pelacakan pengemudi (*live tracking*) dan transaksi pemesanan secara andal. Siklus dimulai dari sisi pengemudi, di mana aplikasi seluler pengemudi (*Driver's App*) secara berkala mengirimkan koordinat lokasi terkini (*real-time GPS data*) melalui koneksi internet dua arah ke **Websocket API** yang berdiri di level regional AWS (di luar VPC). Untuk mencegah lonjakan data masif, Websocket API meneruskan data tersebut ke **Kinesis Data Streams** yang berfungsi sebagai bendungan penahan trafik (*ingestion buffer*). Data koordinat tersebut kemudian diambil secara berkala oleh **AWS Lambda** untuk diproses. Agar AWS Lambda yang berada di luar ini dapat mengirimkan data ke dalam jaringan VPC yang terisolasi, Lambda menembak **VPC Endpoint** (*Interface Type*) yang ditanam di dalam *Private Subnet*. Melalui jalur ini, data diteruskan langsung ke komponen **Driver Service** di dalam *cluster* **Amazon EKS** untuk dilakukan validasi keaktifan akun. Setelah valid, koordinat langsung ditulis ke **Amazon ElastiCache for Redis**, riwayat rute lengkap dicatat ke **Amazon DynamoDB** sebagai log perjalanan, dan status umum pengemudi diperbarui ke database transaksional **Amazon Aurora** secara berkala.

Di sisi lain, ketika seorang penumpang ingin melakukan perjalanan, ia mengirimkan permintaan pemesanan lewat internet yang akan melewati lapisan keamanan **Route 53, Amazon CloudFront, dan AWS WAF** terlebih dahulu, sebelum akhirnya diterima oleh **Application Load Balancer (ALB)**. ALB secara adil membagi beban trafik tersebut ke komponen **Order Service** di dalam *cluster* **Amazon EKS** baik di AZ-1 maupun AZ-2. Order Service bertugas mendaftarkan invoice pesanan baru dengan status awal *TripCreated* ke database utama **Amazon Aurora**, kemudian mempublikasikan sebuah peristiwa (*event*) ke **Amazon EventBridge**. Peristiwa ini langsung ditangkap oleh EventBridge untuk memicu orkestrasi pencarian otomatis menggunakan **AWS Step Functions**. Step Functions bertindak sebagai manajer alur kerja yang memerintahkan **Matching Service** di dalam EKS untuk melakukan pencarian koordinat spasial (*geo-search*) ke **Amazon ElastiCache** guna menarik daftar sepuluh pengemudi terdekat berstatus *available*

dalam radius tertentu. Setelah mendapatkan kandidat terbaik, Step Functions menginstruksikan **Notification Service** untuk menembakkan sinyal penawaran ke ponsel pengemudi terpilih kembali melalui gerbang **Websocket API** di luar VPC.

Siklus pemesanan ditutup oleh respon pengemudi melalui logika *state machine* AWS Step Functions. Jika pengemudi pertama menolak atau *timeout*, sistem otomatis melakukan *looping* untuk mengalihkan penawaran ke pengemudi terdekat berikutnya. Begitu pengemudi menerima pesanan, *looping* berhenti, dan Step Functions memerintahkan **Order Service** memperbarui status menjadi *DriverAssigned* di **Amazon Aurora**. Proses ini memicu **Payment Service** untuk mengunci atau memotong saldo penumpang, serta mengaktifkan **Notification Service**. Terakhir, seluruh riwayat transaksi yang sudah selesai dari Amazon Aurora dan log perjalanan dari Amazon DynamoDB akan diarsipkan secara berkala ke dalam **Amazon S3** sebagai *data lake* jangka panjang untuk keperluan audit dan analisis bisnis. Keamanan seluruh ekosistem dari bencana fisik dijamin oleh *Synchronous Replication* database dari AZ 1 ke AZ 2, serta otomatisasi pengalihan komputasi oleh *EKS Auto Mode* bersama *Horizontal Pod Autoscaler* (HPA) di pusat jaringan.

E. AWS Well-Architected Framework

Berdasarkan AWS Well-Architected Framework, terdapat enam pillars of AWS Well-Architected Framework yang menjadi pedoman dalam merancang arsitektur *cloud*.

1. Operational Excellence (Keunggulan Operasional)

Pilar ini fokus pada bagaimana menjalankan dan memantau sistem untuk memberikan nilai bisnis, serta terus memperbaiki proses di baliknya. Penerapannya ada pada penggunaan Amazon Elastic Kubernetes Service (EKS) yang dikombinasikan dengan Horizontal Pod Autoscaler (HPA) dan EKS Auto Mode. Dengan beralih ke managed services, tim tidak perlu lagi melakukan pemeliharaan server manual seperti patching OS. Selain itu, integrasi Amazon CloudWatch dan AWS CloudTrail membantu memantau kesehatan sistem secara real-time dan otomatis mendeteksi jika ada gangguan operasional.

2. Security (Keamanan)

Pilar ini fokus pada perlindungan informasi, data, sistem, dan aset infrastruktur melalui penilaian risiko dan strategi mitigasi. Penerapannya

dilakukan secara berlapis (Defense-in-Depth) dimana AWS WAF dan Amazon CloudFront di garda terdepan menyaring lalu lintas berbahaya dari internet, sementara jaringan internal diisolasi ketat menggunakan VPC dengan menempatkan EKS dan database di dalam Private Subnet. Manajemen identitas dan hak akses dikendalikan secara granular menggunakan prinsip least privilege melalui AWS IAM, dan kredensial database diamankan secara otomatis oleh AWS Secrets Manager.

3. **Reliability (Keandalan)**

   Pilar ini fokus pada kemampuan sistem untuk menjalankan fungsinya secara tepat dan konsisten, serta cepat pulih dari kegagalan. Penerapannya diwujudkan melalui arsitektur Multi-Availability Zone (Multi-AZ), di mana klaster EKS beserta replika database (Aurora, DynamoDB, dan ElastiCache) diduplikasi secara simetris di dua zona berbeda. Jika Zona 1 mengalami gangguan fisik atau jaringan, Application Load Balancer (ALB) akan langsung mengalihkan lalu lintas ke Zona 2 (failover otomatis). Pendekatan loose coupling menggunakan Amazon EventBridge dan AWS Step Functions juga memastikan gangguan pada satu microservice tidak akan membuat seluruh sistem mati total.

4. **Performance Efficiency (Efisiensi Performa)**

   Pilar ini fokus pada penggunaan sumber daya komputasi dan IT secara efisien untuk memenuhi kebutuhan sistem seiring berkembangnya teknologi dan perubahan beban kerja. Penerapannya dilakukan melalui strategi pemilihan database yang tepat (Polyglot Persistence). Amazon Aurora digunakan untuk pemrosesan transaksi cepat hingga kapasitas 256 TiB, Amazon DynamoDB (NoSQL) dengan teknik denormalisasi data tanpa operasi JOIN menjamin pelacakan lokasi driver secara real-time dengan latensi di bawah 10 milidetik, dan Amazon ElastiCache (memori RAM) digunakan sebagai caching untuk memangkas latensi data statis hingga hitungan mikrodetik.

5. **Cost Optimization (Optimasi Biaya)**

   Pilar ini fokus pada penghindaran biaya operasional yang tidak diperlukan dan memastikan pengeluaran sebanding dengan nilai bisnis. Penerapannya dilakukan melalui analisis kompromi desain (design trade-offs) yang matang. Meskipun biaya infrastruktur untuk arsitektur Multi-AZ dan

replikasi database tergolong mahal, pengeluaran ini dioptimalkan untuk mencegah risiko mati total (downtime). Pada industri ride-hailing, kerugian finansial akibat hilangnya transaksi dan rusaknya kepercayaan pengguna jauh lebih mahal dibanding biaya sewa infrastruktur cadangan, sehingga pengeluaran ini sangat efektif untuk melindungi kelangsungan bisnis.

6. **Sustainability (Keberlanjutan/Ramah Lingkungan)**

   Pilar ini fokus pada minimalisasi dampak lingkungan dari penggunaan layanan cloud, terutama dalam mengurangi konsumsi energi dan emisi karbon. Penerapannya dilakukan dengan menggunakan fitur ElastiCache Serverless dan kapasitas on-demand pada Amazon DynamoDB. Melalui model serverless dan komputasi berbasis kontainer di Amazon EKS, sistem hanya menggunakan daya komputasi dan memori sesuai dengan jumlah penumpang yang aktif. Ketika trafik sepi (misalnya tengah malam), kapasitas sumber daya akan menyusut otomatis hingga ke titik terendah. Hal ini mencegah adanya pemborosan energi dari server yang menyala sia-sia, sehingga arsitektur menjadi jauh lebih ramah lingkungan.

Di samping itu, terdapat pula sepuluh *core best practices*, diantaranya:

1. **Enable Scalability**

   Dalam menghadapi karakteristik industri *ride-hailing* yang memiliki fluktuasi beban kerja (*workload*) sangat ekstrem pada waktu-waktu tertentu, seperti jam berangkat kerja, pulang kantor, atau saat cuaca buruk, arsitektur ini menerapkan prinsip skalabilitas pada setiap lapisannya. Penskalaan horizontal diterapkan secara dinamis melalui kombinasi *Horizontal Pod Autoscaler* (HPA) dan EKS Auto Mode pada *cluster* Amazon EKS. Ketika volume permintaan pemesanan penumpang melonjak tajam, sistem secara otomatis akan melipatgandakan jumlah replika kontainer pada *Order Service* dan *Matching Service* untuk menjaga stabilitas performa aplikasi. Sebaliknya, saat *traffic* kembali normal di luar jam sibuk, infrastruktur akan menyusutkan kapasitas komputasi ke batas minimum secara otomatis. Mekanisme ini memastikan aplikasi tetap responsif dengan latensi rendah, sekaligus mengoptimalkan efisiensi biaya operasional karena pengeluaran komputasi bergerak secara linear mengikuti tren permintaan riil di lapangan.

2. **Automate Your Environment**

Untuk menjamin konsistensi performa dan menghilangkan risiko kesalahan konfigurasi akibat intervensi manual manusia (*human error*), pengelolaan seluruh ekosistem infrastruktur cloud ini dibangun berbasis otomatisasi penuh. Melalui sistem otomatisasi cloud, seluruh elemen jaringan mulai dari penataan Amazon VPC, isolasi *Public* dan *Private Subnet*, konfigurasi gerbang NAT Gateway, hingga regulasi aturan keamanan pada AWS WAF dapat direplikasi secara simetris di dua *Availability Zone* secara instan. Otomatisasi lingkungan ini diperkuat dengan mengintegrasikan Amazon CloudWatch sebagai pusat pemantauan (*monitoring*) dan alarm sistem secara *real-time*. Ketika CloudWatch mendeteksi adanya anomali performa atau lonjakan beban kerja melebihi ambang batas yang ditentukan, sistem secara otomatis akan memicu tindakan korektif, seperti eksekusi tindakan otomatis untuk pemulihan jaringan atau penambahan kapasitas, tanpa menunggu tindakan manual dari tim operasional. Bagi platform komersial dengan mobilitas tinggi seperti *ride-hailing*, standardisasi lingkungan terotomatisasi yang adaptif ini menjadi sangat krusial tidak hanya untuk menjaga kestabilan aplikasi secara mandiri (*self-healing*), tetapi juga untuk meminimalkan waktu pemulihan (*Recovery Time Objective*) jika sewaktu-waktu terjadi gangguan.

3. **Treat Resource as Disposable**

Di dalam *cluster* Amazon EKS, kontainer *microservices* maupun *worker nodes* tidak dianggap sebagai aset permanen yang harus diperbaiki atau didiagnosis secara manual ketika mengalami gangguan operasional (*freeze* atau *error*). Jika suatu *instance* kontainer terdeteksi mengalami penurunan performa atau gagal merespons *health check*, sistem tidak akan melakukan *troubleshooting* di lingkungan produksi, melainkan langsung menghancurkan (*terminate*) kontainer bermasalah tersebut seketika itu juga. *Cluster* EKS kemudian akan langsung meluncurkan replika kontainer baru yang benar-benar bersih dan sehat. Pendekatan ini memastikan kelangsungan transaksi pemesanan pengguna tetap berjalan mulus tanpa interupsi, sementara isolasi terhadap kegagalan komponen dapat diselesaikan secara otomatis di balik layar.

4. **Use Loosely Coupled Component**

Sistem ride-hailing ini mengimplementasikan prinsip *loosely coupled components* sesuai dengan *best practice* arsitektur cloud untuk menghindari dependensi antar-layanan yang kaku. Komunikasi antar-microservice (seperti Order Service, Matching Service, dan Payment Service) diatur menggunakan kombinasi Amazon EventBridge dan AWS Step Functions yang bertindak sebagai pengatur alur kerja (*workflow orchestrator*). Melalui pendekatan arsitektur berbasis peristiwa (*event-driven*) ini, setiap layanan dapat beroperasi secara asinkron dan mandiri. Dampaknya, apabila salah satu komponen mengalami gangguan fungsional, misalnya Notification Service mengalami kelambatan, alur transaksi utama seperti pemesanan dan pencarian driver tetap dapat berjalan normal tanpa memicu kegagalan sistem secara beruntun (*cascading failure*).

5. **Design Service, Not Server**

Dalam pengelolaan infrastruktur komputasi, sistem beralih dari manajemen server tradisional ke pengelolaan berbasis layanan terkelola (*managed services*) dengan memanfaatkan Amazon Elastic Kubernetes Service (EKS). Ditambah dengan fitur Horizontal Pod Autoscaler (HPA) dan EKS Auto Mode, tim pengembang tidak perlu lagi memikirkan pemeliharaan sistem operasi, patching, atau konfigurasi server Amazon EC2 secara manual. Hal ini memungkinkan tim untuk sepenuhnya fokus pada pengembangan logika bisnis microservice, sementara replikasi, ketersediaan kontainer, dan penskalaan otomatis saat lonjakan beban penumpang ditangani langsung oleh platform AWS.

6. **Choose the Right Database Solution**

Mengingat aplikasi ride-hailing memiliki karakteristik beban kerja (*workload*) yang kompleks dengan kebutuhan latensi dan struktur data yang bervariasi, arsitektur ini menerapkan strategi Polyglot Persistence dengan mencocokkan teknologi database terhadap kebutuhan spesifik layanannya. Pada skema infrastruktur, beban penyimpanan dibagi ke dalam tiga solusi database terkelola AWS yang berbeda. Amazon Aurora digunakan sebagai database relasional untuk menjamin konsistensi tingkat tinggi (ACID) pada data transaksional seperti riwayat pembayaran. Sementara itu, Amazon DynamoDB (NoSQL) dipilih untuk menangani data dengan throughput tinggi

dan kebutuhan latensi berskala milidetik, seperti pencatatan koordinat lokasi driver secara real-time. Terakhir, Amazon ElastiCache diintegrasikan sebagai in-memory database untuk mengoptimalkan performa pembacaan data.

7. **Avoid Single Points of Failure**

   Arsitektur sistem ini dibangun dengan memegang prinsip dasar bahwa setiap komponen infrastruktur memiliki potensi untuk gagal (*assume everything fails*), sehingga eliminasi titik kegagalan tunggal menjadi prioritas utama. Mitigasi risiko ini dilakukan melalui strategi redundansi penuh berbasis arsitektur Multi-Availability Zone (Multi-AZ) di dalam Amazon VPC. Komponen microservice pada Amazon EKS serta lapisan penyimpanan data (Amazon Aurora, DynamoDB, dan ElastiCache) direplikasikan secara simetris di dua zona ketersediaan yang terisolasi, yaitu Availability Zone 1 (sebagai primary) dan Availability Zone 2 (sebagai replica/standby). Jika pusat data pada AZ 1 mengalami pemadaman total atau gangguan jaringan, Application Load Balancer akan segera mengalihkan seluruh lalu lintas pengguna dan database akan melakukan failover otomatis ke AZ 2, sehingga ketersediaan layanan tetap terjaga tanpa interupsi.

8. **Optimize for Cost**

   Menerapkan redundansi Multi-AZ dan replikasi database berskala besar tentu menimbulkan konsekuensi berupa peningkatan biaya operasional infrastruktur. Namun, keputusan arsitektur ini diambil berdasarkan evaluasi kompromi desain (*design trade-offs*) yang matang antara aspek finansial dan toleransi risiko bisnis. Bagi platform digital dengan mobilitas tinggi seperti *ride-hailing*, biaya kerugian finansial yang timbul akibat waktu henti layanan (*downtime*) jauh lebih besar dan fatal dibandingkan biaya investasi untuk menjaga redundansi. Kerugian tersebut tidak hanya dihitung dari hilangnya potensi transaksi secara langsung, tetapi juga dari risiko jangka panjang seperti penurunan tingkat kepercayaan pengguna (*user trust*) yang dapat berpindah ke kompetitor. Oleh karena itu, pengeluaran biaya dalam arsitektur ini dioptimalkan secara strategis dengan memprioritaskan ketersediaan tinggi (*high availability*) demi kelangsungan bisnis.

9. **Use Caching**

   Untuk meningkatkan efisiensi performa dan meminimalkan operasi pengambilan data yang redundan ke database utama, sistem ini

mengintegrasikan Amazon ElastiCache pada lapisan penyimpanan data. Mekanisme caching ini dimanfaatkan untuk menyimpan data yang memiliki karakteristik *read-heavy* atau sering diakses oleh aplikasi namun jarang berubah, seperti informasi profil dasar, ketentuan tarif wilayah, serta data promo aktif. Dengan mengambil data langsung dari memori (*in-memory storage*) berskala sub-milidetik, sistem tidak perlu melakukan query terstruktur yang berat secara berulang ke Amazon Aurora. Implementasi ini secara signifikan memangkas latensi aplikasi pada sisi pengguna akhir sekaligus meringankan beban kerja database transaksional utama, sehingga kapasitas komputasi database dapat dialokasikan penuh untuk memproses penulisan transaksi pemesanan baru yang krusial.

10. Secure Your Entire Infrastructure

Keamanan pada sistem *ride-hailing* ini dirancang secara menyeluruh dengan membangun proteksi di setiap lapisan infrastruktur (*Defense-in-Depth*) guna melindungi aplikasi, data, serta integritas aktivitas pengguna. Komponen AWS WAF dan Amazon CloudFront ditempatkan di garda depan untuk menyaring lalu lintas siber berbahaya dari internet publik. Pada tingkat jaringan, segmentasi dilakukan secara ketat menggunakan komponen VPC dengan mengisolasi klaster EKS dan database di dalam Private Subnet agar tidak dapat diakses langsung dari luar. Pengaturan hak akses operasional dikendalikan menggunakan asas *least privilege* melalui AWS IAM dan AWS Secrets Manager untuk melindungi informasi sensitif. Seluruh aktivitas dan kesehatan sistem ini dipantau secara real-time oleh Amazon CloudWatch serta dicatat jejak auditnya melalui AWS CloudTrail demi pemenuhan standar kepatuhan keamanan.

BAB IV USE CASE

A. Driver Menerima Pesanan

Seorang penumpang membuat permintaan perjalanan. Sistem berhasil menemukan pengemudi terdekat dan pengemudi menerima permintaan perjalanan yang dikirimkan sistem. Proses yang terjadi pada sistem adalah, penumpang membuat permintaan perjalanan melalui aplikasi. Permintaan tersebut melewati Route 53, kemudian diteruskan ke Amazon CloudFront yang sudah terintegrasi dengan AWS WAF untuk menjaga keamanan. Setelah melewati Amazon CloudFront, permintaan akan masuk ke Application Load Balancer dan diteruskan ke Order Service pada Amazon EKS. Order Service akan membuat data perjalanan baru dengan status *TripCreated* dan menyimpannya ke Amazon Aurora. Order Service mengirimkan *event* ke Amazon EventBridge yang memicu *workflow* AWS Step Functions. Step Function memanggil Matching Service untuk melalui *geo-search* ke Amazon ElastiCache Redis untuk mencari sepuluh pengemudi terdekat berdasarkan status yang *available* dalam radius tertentu. Setelah pengemudi terbaik dipilih, Step Function memanggil Notification Service untuk mengirimkan penawaran perjalanan ke *Driver's App* melalui WebSocket API. Pengemudi menekan tombol "Accept" pada *Driver's App*. Respon tersebut dikirim melalui WebSocket API kemudian diteruskan ke Kinesis Data Stream, AWS Lambda, dan VPC Endpoint, lalu Notification Service. Notification Service mengirimkan hasil ke Step Functions. State Machine pada Step Function berada pada state "DriverAccepted". *Workflow* berhenti melakukan pencarian pengemudi. Kemudian, Step Functions memanggil Order Service untuk memperbarui status perjalanan menjadi *DriverAssigned*, menyimpannya ke Amazon Aurora. Driver Service akan mengubah status pengemudi menjadi "Busy" ke ElastiCache dan Aurora. Disaat yang bersamaan, Step Functions memanggil Payment Service untuk melakukan *pre-authorization* saldo atau mengunci dana sementara. Selama perjalanan menuju lokasi penjemputan, *Driver's App* mengirimkan lokasi secara *real-time*. Driver Service menyimpan posisi terbaru ke Amazon ElastiCache dan menyimpan histori perjalanan ke Amazon DynamoDB. Setelah perjalanan selesai, status berubah menjadi *TripCompleted* dan Step Function memanggil Payment Service untuk memotong saldo sesuai dengan nominal perjalanan. Status transaksi diperbarui di Amazon Aurora, histori perjalanan disimpan di DynamoDB dan data diarsipkan ke Amazon S3.

B. Driver Menolak Pesanan

Seorang penumpang membuat permintaan perjalanan. Sistem berhasil menemukan pengemudi terdekat. Namun, pengemudi tersebut menolak pesanan atau tidak memberikan respons hingga batas waktu yang ditentukan (*timeout*). Proses yang terjadi pada sistem adalah, penumpang membuat permintaan perjalanan melalui aplikasi. Permintaan tersebut melewati Route 53, kemudian diteruskan ke Amazon CloudFront yang sudah terintegrasi dengan AWS WAF untuk menjaga keamanan. Setelah melewati Amazon CloudFront, permintaan akan masuk ke Application Load Balancer dan diteruskan ke Order Service pada Amazon EKS. Order Service akan membuat data perjalanan baru dengan status *TripCreated* dan menyimpannya ke Amazon Aurora. Order Service mengirimkan *event* ke Amazon EventBridge yang memicu *workflow* AWS Step Functions. Step Function memanggil Matching Service untuk melalui *geo-search* ke Amazon ElastiCache Redis untuk mencari sepuluh pengemudi terdekat berdasarkan status yang *available* dalam radius tertentu. Setelah pengemudi terbaik dipilih, Step Function memanggil Notification Service untuk mengirimkan penawaran perjalanan ke *Driver's App* melalui WebSocket API. Pengemudi tersebut menekan tombol "Reject" atau tidak merespons hingga *timeout*. Respon penolakan dikirim kembali melalui WebSocket API kemudian diteruskan ke Kinesis Data Stream, AWS Lambda, dan VPC Endpoint, lalu Notification Service. Notification Service mengirim status penolakan ke Step Functions sehingga State Machine Step Function berada pada "Driver Rejected". *Workflow* melakukan *looping* dan kembali memanggil Matching Service. Matching Service memilih pengemudi berikutnya dari daftar kandidat. Kemudian, Notification Service mengirimkan penawaran baru kepada pengemudi kedua. Proses tersebut berulang hingga pengemudi menerima pesanan atau seluruh kandidat habis. Jika tidak ada pengemudi yang menerima, maka status pada Order Service menjadi *TripCancelled* dan disimpan ke Amazon Aurora. Lalu, penumpang akan menerima notifikasi bahwa tidak ada pengemudi yang tersedia.

C. Peak Hour Traffic

Pada jam sibuk seperti berangkat kerja (pukul 07.00–09.00) atau pulang kerja (pukul 16.00–19.00), jumlah pengemudi dan penumpang meningkat secara drastis. Misalnya pada jam normal hanya terdapat 5.000 permintaan/menit sedangkan pada jam sibuk bisa mencapai 50.000 permintaan/menit. Tanpa adanya *autoscaling*, sistem

berpotensi mengalami CPU overload, memory overload, latensi tinggi, dan *request timeout*. Oleh karena itu, arsitektur ini memiliki sistem *autoscaling* untuk mencegah terjadinya potensi tersebut.

Pada jam sibuk, banyak penumpang melakukan pemesanan secara bersamaan. Permintaan perjalanan masuk melalui Route 53 kemudian diteruskan ke Amazon CloudFront yang diintegrasi dengan AWS WAF dan diteruskan ke ALB. ALB mendistribusikan *traffic* ke pod Order Service. Misalnya target CPU adalah 70% sedangkan saat *traffic* meningkat, CPU pada Order Service bisa mencapai 85%, Matching Service 90%, Notification Service 88%. *Horizontal Pod Autoscaler* (HPA) akan mendeteksi bahwa target telah terlampaui. HPA kemudian membuat replika pod baru untuk *microservice* yang mengalami peningkatan beban sehingga permintaan pengguna dapat dibagi ke lebih banyak instance. Sebagai contoh, jumlah pod pada Order Service dapat meningkat dari 4 pod menjadi 12 pod, sementara Matching Service dan Notification Service juga memperoleh tambahan pod sesuai kebutuhan. Dengan mekanisme ini, sistem mampu mempertahankan waktu respons yang rendah meskipun jumlah permintaan meningkat drastis. Penambahan pod oleh HPA membutuhkan kapasitas komputasi tambahan pada *cluster* Kubernetes. Oleh karena itu, Amazon EKS Auto Mode secara otomatis menambahkan worker node baru ketika node yang ada tidak lagi memiliki kapasitas CPU atau memori yang cukup. EKS Auto Mode akan memilih jenis instance yang sesuai, membuat node baru, dan mendistribusikannya ke Availability Zone yang tersedia untuk menjaga keseimbangan beban dan ketersediaan sistem.

Di saat yang sama, Driver Service menerima pembaruan lokasi dari ribuan pengemudi melalui WebSocket API, Amazon Kinesis Data Streams, AWS Lambda, dan VPC Endpoint. Data lokasi tersebut disimpan pada Amazon ElastiCache untuk kebutuhan pencarian pengemudi terdekat secara cepat, sementara riwayat perjalanan dicatat ke Amazon DynamoDB. Order Service dan Payment Service tetap menggunakan Amazon Aurora untuk menangani transaksi yang memerlukan konsistensi tinggi. Pembagian layanan database ini mencegah terjadinya *bottleneck* pada satu database ketika volume transaksi meningkat pada jam sibuk.

Setelah jam sibuk berakhir dan jumlah permintaan menurun, penggunaan CPU dan memori pada pod kembali ke tingkat normal. HPA melakukan *scale-down* dengan mengurangi jumlah pod yang tidak lagi diperlukan. Kemudian EKS Auto Mode juga otomatis menghapus node yang tidak digunakan sehingga kapasitas kembali normal.

BAB V KESIMPULAN

Berdasarkan hasil perancangan arsitektur cloud untuk *Ride-Hailing Dispatch System* berbasis Amazon Web Services (AWS), dapat disimpulkan bahwa rancangan yang diusulkan mampu menjawab kebutuhan sistem dalam menangani pengiriman data GPS secara *real-time* dengan latensi rendah. Hal ini dicapai melalui pemanfaatan Amazon API Gateway WebSocket APIs, Amazon Kinesis Data Streams, AWS Lambda, Amazon ElastiCache, dan Amazon DynamoDB yang memungkinkan data lokasi pengemudi diproses secara *real-time* serta mendukung pencarian *driver* terdekat dengan respons yang cepat.

Selain itu, kebutuhan akan *high availability* dan *fault tolerance* dipenuhi melalui penerapan arsitektur *Single-Region Multi-Availability Zone* (*Multi-AZ*). Layanan seperti Amazon EKS, Amazon Aurora, Amazon DynamoDB, dan Amazon ElastiCache didistribusikan ke beberapa Availability Zone sehingga sistem tetap dapat beroperasi meskipun terjadi kegagalan pada salah satu zona. Dukungan Application Load Balancer (ALB), Route 53 Health Check, *failover* otomatis, dan replikasi data turut meningkatkan keandalan serta kontinuitas layanan.

Kebutuhan untuk mengakomodasi lonjakan *workload* juga dapat dipenuhi melalui mekanisme *autoscaling* pada Amazon EKS, yaitu dengan Horizontal Pod Autoscaler (HPA) dan EKS Auto Mode yang secara otomatis menyesuaikan kapasitas sesuai beban kerja. Secara keseluruhan, rancangan arsitektur *cloud* yang diusulkan telah memenuhi kebutuhan fungsional dan non-fungsional sistem *ride-hailing*, mencakup aspek real-time *processing, scalability, availability, reliability, security,* dan *cost optimization*, sehingga mampu mendukung operasional layanan secara andal, aman, dan efisien.

DAFTAR PUSTAKA

Amazon Web Services. (n.d.). *Amazon API Gateway WebSocket APIs*. https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-websocket-api.html

Amazon Web Services. (n.d.). *Amazon Aurora overview*. https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/CHAP_AuroraOverview.html

Amazon Web Services. (n.d.). *Amazon CloudFront introduction*. https://docs.aws.amazon.com/id_id/AmazonCloudFront/latest/DeveloperGuide/Introduction.html

Amazon Web Services. (n.d.). *Amazon CloudWatch overview*. https://docs.aws.amazon.com/id_id/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html

Amazon Web Services. (n.d.). *Amazon DynamoDB introduction*. https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html

Amazon Web Services. (n.d.). *Amazon EC2 regions and availability zones*. https://docs.aws.amazon.com/id_id/AWSEC2/latest/UserGuide/using-regions-availability-zones.html#concepts-availability-zones

Amazon Web Services. (n.d.). *Amazon ElastiCache overview*. https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/WhatIs.html

Amazon Web Services. (n.d.). *Amazon Elastic Kubernetes Service*. In *Overview of deployment options whitepaper*. https://docs.aws.amazon.com/id_id/whitepapers/latest/overview-deployment-options/amazon-elastic-kubernetes-service.html

Amazon Web Services. (n.d.). *Amazon Kinesis Data Streams developer guide*. https://docs.aws.amazon.com/streams/latest/dev/introduction.html

Amazon Web Services. (n.d.). *Amazon Route 53*. https://aws.amazon.com/id/route53/

Amazon Web Services. (n.d.). *AWS CloudTrail user guide*. https://docs.aws.amazon.com/id_id/awscloudtrail/latest/userguide/cloudtrail-user-guide.html

Amazon Web Services. (n.d.). *AWS IAM introduction*. https://docs.aws.amazon.com/id_id/IAM/latest/UserGuide/introduction.html

Amazon Web Services. (n.d.). *AWS services that work with IAM*. https://docs.aws.amazon.com/id_id/IAM/latest/UserGuide/reference_aws-services-that-work-with-iam.html

Amazon Web Services. (n.d.). *AWS Lambda developer guide*. https://docs.aws.amazon.com/lambda/latest/dg/welcome.html

Amazon Web Services. (n.d.). *AWS Secrets Manager user guide*. https://docs.aws.amazon.com/id_id/secretsmanager/latest/userguide/intro.html

Amazon Web Services. (n.d.). *AWS Step Functions workflows*. https://docs.aws.amazon.com/id_id/wellarchitected/latest/serverless-applications-lens/step-functions-workflows.html

Amazon Web Services. (n.d.). *AWS Well-Architected Framework: The six pillars*. https://aws.amazon.com/blogs/apn/the-6-pillars-of-the-aws-well-architected-framework/

Amazon Web Services. (n.d.). *Internet gateways for Amazon VPC*. https://docs.aws.amazon.com/id_id/vpc/latest/userguide/VPC_Internet_Gateway.html

Amazon Web Services. (n.d.). *NAT gateways*. https://docs.aws.amazon.com/id_id/vpc/latest/userguide/vpc-nat-gateway.html

Amazon Web Services. (n.d.). *What is Amazon EC2 Auto Scaling?* https://docs.aws.amazon.com/id_id/eks/latest/best-practices/automode.html

Amazon Web Services. (n.d.). *What is Amazon S3?* https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html

Amazon Web Services. (n.d.). *What is Amazon VPC?* https://docs.aws.amazon.com/id_id/vpc/latest/userguide/what-is-amazon-vpc.html

Amazon Web Services. (n.d.). *What is AWS WAF?* https://docs.aws.amazon.com/waf/latest/developerguide/waf-chapter.html

AWS Partner Network. (n.d.). *How to use interface VPC endpoints to meet your security objectives*. https://aws.amazon.com/blogs/security/how-to-use-interface-vpc-endpoints-to-meet-your-security-objectives/

Cloud Forecast. (n.d.). *ECS vs EKS: Which AWS container service should you choose?* https://www.cloudforecast.io/ecs-vs-eks/

Kubernetes. (n.d.). *Horizontal Pod Autoscaling*. https://kubernetes.io/id/docs/tasks/run-application/horizontal-pod-autoscale/

ProjectPro. (n.d.). *Amazon Kinesis: Architecture, components, and use cases*. https://www.projectpro.io/article/amazon-kinesis/818